UNIVERSIDAD ANDINA DEL CUSCO

FACULTAD DE INGENIERÍA Y ARQUITECTURA

ASIGNATURA: Modelado de Bases de Datos

ACTIVIDAD : Implementación de Modelos Físicos

Escuela profesional:

Ingeniería de Sistemas

DOCENTE:

Espetia Huamanga Hugo

GRUPO:

No DATA

ESTUDIANTE:

Olivera Ochoa Paul Claus - 024100569i (SM) - (100%)

Carpio Delgado, Alejandro Eduardo - 024100051j - (100%)

Haro Rosales, Joaquín - 024100970e - (100%)

Molero Zegarra Thiago Valentino - 024100247a - (100%)

Cusco – Perú

Marzo del 2026

1. INTRODUCCIÓN

El presente informe forma parte de la asignatura de Modelado de Bases de Datos de la Universidad Andina del Cusco. En este trabajo documentamos el paso de los diagramas de modelo físico a código SQL real.

Para entender cómo varía el código entre los diferentes programas que existen en el mercado, nuestro grupo programó la creación de las tablas y relaciones para siete sistemas distintos utilizando cuatro de los gestores de bases de datos más conocidos: PostgreSQL, Oracle, SQL Server y MySQL. Este ejercicio comparativo nos permite ver que, aunque la lógica del diseño (las llaves primarias y foráneas) es la misma, la forma de escribir el código cambia dependiendo del programa que se utilice.

2. OBJETIVOS

Objetivo General:

- **Crear los scripts de código SQL (Lenguaje de Definición de Datos - DDL) para siete sistemas de bases de datos diferentes, asegurando que las tablas y sus relaciones funcionen correctamente en distintos gestores de base de datos.**

Objetivos Específicos:

- **Traducir los diagramas físicos, respetando las cardinalidades (relaciones de uno a muchos) y pasando correctamente las llaves primarias (PRIMARY KEY) y foráneas (FOREIGN KEY).**
- **Comparar cómo se escriben los tipos de datos y los campos autoincrementales en PostgreSQL, Oracle, SQL Server y MySQL.**
- **Aplicar buenas prácticas al codificar, como eliminar las tablas existentes antes de crearlas de nuevo, para evitar errores en la base de datos al momento de ejecutar el código.**

3. DESARROLLO DE LOS MODELOS FÍSICOS

Para este trabajo resolvimos siete casos de estudio. En todos los sistemas tuvimos mucho cuidado con el orden de creación de las tablas: primero creamos las tablas independientes (las que no tienen llaves foráneas) y al final las tablas transaccionales (las que dependen de otras), para que el programa no nos dé errores de referencia.

Los 7 sistemas que pasamos a código en los 4 gestores son:

1. **Sistema "Super Giros": Registro de transacciones bancarias, cuentas y clientes (incluye una relación recursiva).**
2. **Sistema del INPE: Control de turnos y asistencias de los empleados.**
3. **Sistema pollería "Don Gallino": Gestión de ventas, clientes, productos y comprobantes.**
4. **Sistema "Electro Track": Control de almacén de electrodomésticos con llaves foráneas opcionales (para diferenciar ingresos y salidas).**

5. Sistema "Mac Salud": Asistencia de enfermeras, conectando calendarios y ambientes médicos.

6. Sistema pollería "Los Ángeles": Sistema de facturación electrónica con integridad referencial estricta.

7. Sistema gimnasio "Titanium": Control de asistencia de clientes y grupos de disciplinas.

ORACLE

2.- Sistema de registro de transacciones bancarias a nivel nacional - Perú de la empresa Super Giros.

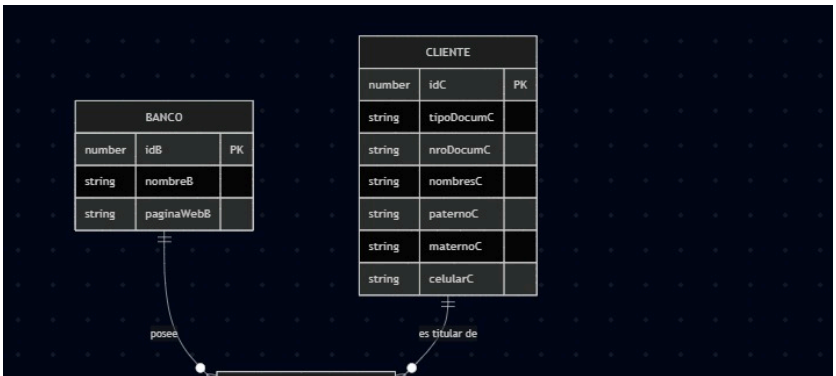
Código

```
BEGIN
  EXECUTE IMMEDIATE 'DROP TABLE Operacion_SG CASCADE CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE Cuenta_SG CASCADE CONSTRAINTS';
```

```
    EXECUTE IMMEDIATE 'DROP TABLE Banco_SG CASCADE CONSTRAINTS';
    EXECUTE IMMEDIATE 'DROP TABLE Empleado_SG CASCADE
CONSTRAINTS';
    EXECUTE IMMEDIATE 'DROP TABLE Cliente_SG CASCADE CONSTRAINTS';
  EXCEPTION WHEN OTHERS THEN NULL;
  END;
  /

  -- 1. Tabla Banco
  CREATE TABLE Banco_SG (
    idB         NUMBER(10) PRIMARY KEY,
    nombreB     VARCHAR2(100) NOT NULL,
    paginaWebB  VARCHAR2(100)
  );

  -- 2. Tabla Cliente (Atributos: idC, tipoDocumC, nroDocumC, nombresC, paternoC,
maternoC, celularC)
  CREATE TABLE Cliente_SG (
    idC         NUMBER(10) PRIMARY KEY,
    tipoDocumC  VARCHAR2(20),
    nroDocumC   VARCHAR2(20),
    nombresC    VARCHAR2(100),
    paternoC    VARCHAR2(100),
```

```
    maternoC     VARCHAR2(100),
    celularC     VARCHAR2(20)
);

-- 3. Tabla Cuenta (Atributos: idCuenta, nroCuenta, saldoCuenta)
CREATE TABLE Cuenta_SG (
    idCuenta     NUMBER(10) PRIMARY KEY,
    idB          NUMBER(10) NOT NULL,
    idC          NUMBER(10) NOT NULL,
    nroCuenta    VARCHAR2(50) UNIQUE,
    saldoCuenta  NUMBER(15, 2),
    CONSTRAINT fk_banco_cta_sg FOREIGN KEY (idB) REFERENCES
Banco_SG(idB),
    CONSTRAINT fk_cliente_cta_sg FOREIGN KEY (idC) REFERENCES
Cliente_SG(idC)
);

-- 4. Tabla Empleado (Atributos: idE, tipoDocumE, nroDocumE, nombresE, paternoE,
maternoE, celularE)
CREATE TABLE Empleado_SG (
    idE          NUMBER(10) PRIMARY KEY,
    tipoDocumE   VARCHAR2(20),
    nroDocumE    VARCHAR2(20),
```

```
    nombresE     VARCHAR2(100),
    paternoE     VARCHAR2(100),
    maternoE     VARCHAR2(100),
    celularE     VARCHAR2(20)
);

-- 5. Tabla Operacion (Atributos: idOpe, tipoOpe, fechaHoraOpe, montoOpe,
comisionOpe)
CREATE TABLE Operacion_SG (
    idOpe         NUMBER(10) PRIMARY KEY,
    idCuenta      NUMBER(10) NOT NULL,
    idE           NUMBER(10) NOT NULL,
    tipoOpe       VARCHAR2(50),
    fechaHoraOpe  DATE DEFAULT SYSDATE,
    montoOpe      NUMBER(12, 2),
    comisionOpe   NUMBER(12, 2),
    CONSTRAINT fk_cta_ope_sg FOREIGN KEY (idCuenta) REFERENCES
Cuenta_SG(idCuenta),
    CONSTRAINT fk_emp_ope_sg FOREIGN KEY (idE) REFERENCES
Empleado_SG(idE)
)
```



3.- Sistema de registro de asistencia del personal del INPE.

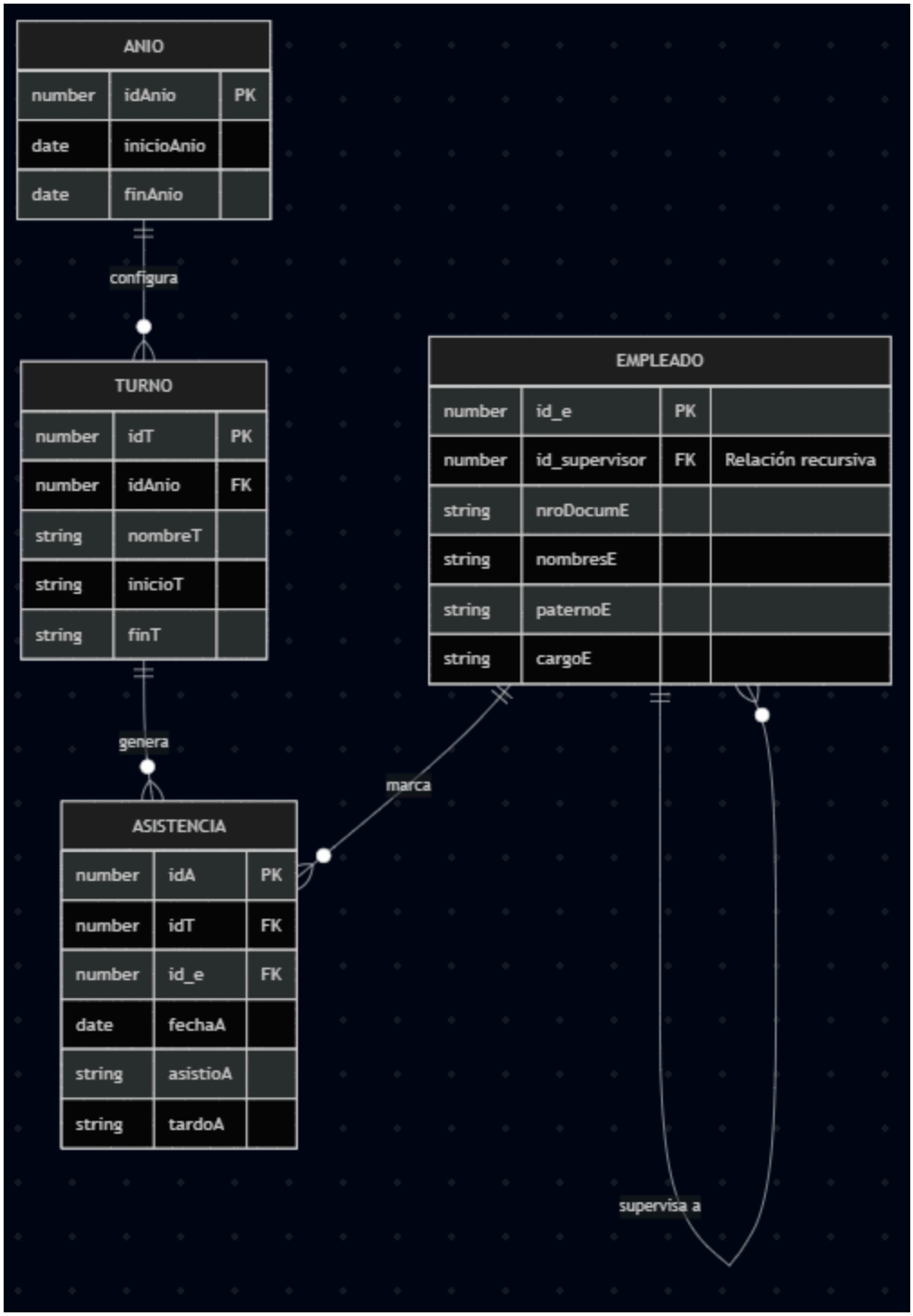
Código

```
BEGIN
  EXECUTE IMMEDIATE 'DROP TABLE EH_Detalle_Movimiento CASCADE
CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE EH_Comprobante CASCADE
CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE EH_Orden_Compra CASCADE
CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE EH_Electrodomestico CASCADE
CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE EH_Cliente CASCADE CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE EH_Proveedor CASCADE
CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/


-- 1. Tabla Proveedor
CREATE TABLE EH_Proveedor (
  idP          NUMBER(10) PRIMARY KEY,
  rucP         VARCHAR2(20) UNIQUE,
  razonSocialP  VARCHAR2(100),
```

```
    direccionP    VARCHAR2(150),
    celularP      VARCHAR2(20)
);

-- 2. Tabla Cliente
CREATE TABLE EH_Cliente (
    idC           NUMBER(10) PRIMARY KEY,
    tipoDocumC    VARCHAR2(20),
    nroDocumC     VARCHAR2(20) UNIQUE,
    nombresC      VARCHAR2(100),
    paternoC      VARCHAR2(100),
    maternoC      VARCHAR2(100),
    celularC      VARCHAR2(20)
);

-- 3. Tabla Electrodomestico
CREATE TABLE EH_Electrodomestico (
    idE           NUMBER(10) PRIMARY KEY,
    nombreE       VARCHAR2(100),
    marcaE        VARCHAR2(50),
    modeloE       VARCHAR2(50),
    precioVentaE  NUMBER(10, 2),
    precioCompraE NUMBER(10, 2),
```

```
    stockE        NUMBER(10)
);

-- 4. Tabla Orden de Compra (Relación con Proveedor)
CREATE TABLE EH_Orden_Compra (
    idOrden       NUMBER(10) PRIMARY KEY,
    idP           NUMBER(10) NOT NULL,
    fechaHoraOrd  DATE DEFAULT SYSDATE,
    totalOrden    NUMBER(12, 2),
    CONSTRAINT fk_eh_prov_ord FOREIGN KEY (idP) REFERENCES
EH_Proveedor(idP)
);

-- 5. Tabla Comprobante (Relación con Cliente)
CREATE TABLE EH_Comprobante (
    idComp        NUMBER(10) PRIMARY KEY,
    idC           NUMBER(10) NOT NULL,
    tipoComp      VARCHAR2(20), -- Boleta o Factura
    fechaHoraComp DATE DEFAULT SYSDATE,
    totalComp     NUMBER(12, 2),
    CONSTRAINT fk_eh_cli_comp FOREIGN KEY (idC) REFERENCES
EH_Cliente(idC)
);
```

```
-- 6. Tabla Detalle de Movimiento (Entradas/Salidas)
-- Esta tabla une los productos con las órdenes (compras) o comprobantes (ventas)
CREATE TABLE EH_Detalle_Movimiento (
    idDet        NUMBER(10) PRIMARY KEY,
    idE          NUMBER(10) NOT NULL,
    idOrden      NUMBER(10), -- Puede ser nulo si es una venta
    idComp       NUMBER(10), -- Puede ser nulo si es una compra
    tipoDet      VARCHAR2(20), -- 'ENTRADA' o 'SALIDA'
    cantidadDet  NUMBER(10),
    subtotalDet  NUMBER(10, 2),
    CONSTRAINT fk_eh_prod_det FOREIGN KEY (idE) REFERENCES
EH_Electrodomestico(idE),
    CONSTRAINT fk_eh_ord_det  FOREIGN KEY (idOrden) REFERENCES
EH_Orden_Compra(idOrden),
    CONSTRAINT fk_eh_comp_det FOREIGN KEY (idComp) REFERENCES
EH_Comprobante(idComp)
);
```



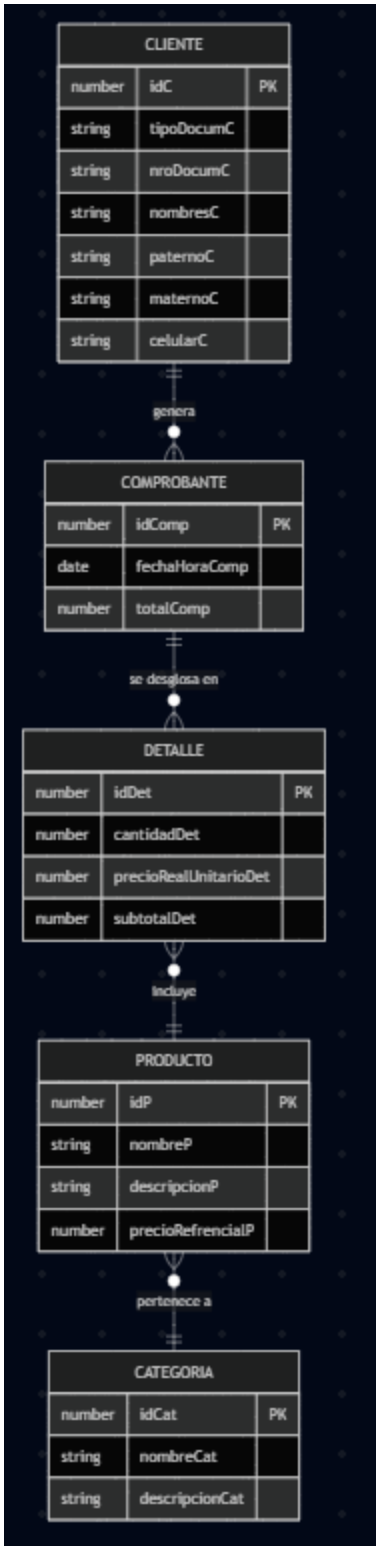
4.- Sistema de registro de venta de platillos y bebidas en la pollería "Don Gallino"

Código

```
-- LIMPIEZA INICIAL
BEGIN
  EXECUTE IMMEDIATE 'DROP TABLE DG_Detalle CASCADE CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE DG_Comprobante CASCADE
CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE DG_Producto CASCADE CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE DG_Categoria CASCADE
CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE DG_Cliente CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

-- 1. Categoría
CREATE TABLE DG_Categoria (
   idCat         NUMBER(10) PRIMARY KEY,
   nombreCat     VARCHAR2(100),
   descripcionCat VARCHAR2(255)
);

-- 2. Producto
CREATE TABLE DG_Producto (
   idP             NUMBER(10) PRIMARY KEY,
```

```
    idCat            NUMBER(10) NOT NULL,
    nombreP          VARCHAR2(100),
    descripcionP     VARCHAR2(255),
    precioRefrencialP NUMBER(10, 2),
    CONSTRAINT fk_dg_cat_prod FOREIGN KEY (idCat) REFERENCES
DG_Categoria(idCat)
);

-- 3. Cliente
CREATE TABLE DG_Cliente (
    idC          NUMBER(10) PRIMARY KEY,
    tipoDocumC   VARCHAR2(20),
    nroDocumC    VARCHAR2(20) UNIQUE,
    nombresC     VARCHAR2(100),
    paternoC     VARCHAR2(100),
    maternoC     VARCHAR2(100),
    celularC     VARCHAR2(20)
);

-- 4. Comprobante
CREATE TABLE DG_Comprobante (
    idComp        NUMBER(10) PRIMARY KEY,
    idC           NUMBER(10) NOT NULL,
```

```
    fechaHoraComp  DATE DEFAULT SYSDATE,
    totalComp      NUMBER(12, 2),
    CONSTRAINT fk_dg_cli_comp FOREIGN KEY (idC) REFERENCES
DG_Cliente(idC)
);

-- 5. Detalle (Tabla de unión entre Comprobante y Producto)
CREATE TABLE DG_Detalle (
    idDet                NUMBER(10) PRIMARY KEY,
    idComp               NUMBER(10) NOT NULL,
    idP                  NUMBER(10) NOT NULL,
    cantidadDet          NUMBER(10),
    precioRealUnitarioDet  NUMBER(10, 2),
    subtotalDet          NUMBER(10, 2),
    CONSTRAINT fk_dg_comp_det FOREIGN KEY (idComp) REFERENCES
DG_Comprobante(idComp),
    CONSTRAINT fk_dg_prod_det FOREIGN KEY (idP) REFERENCES
DG_Producto(idP)
);
```



5.- Sistema de registro de entrada y salida de electrodomésticos “Electro Track”

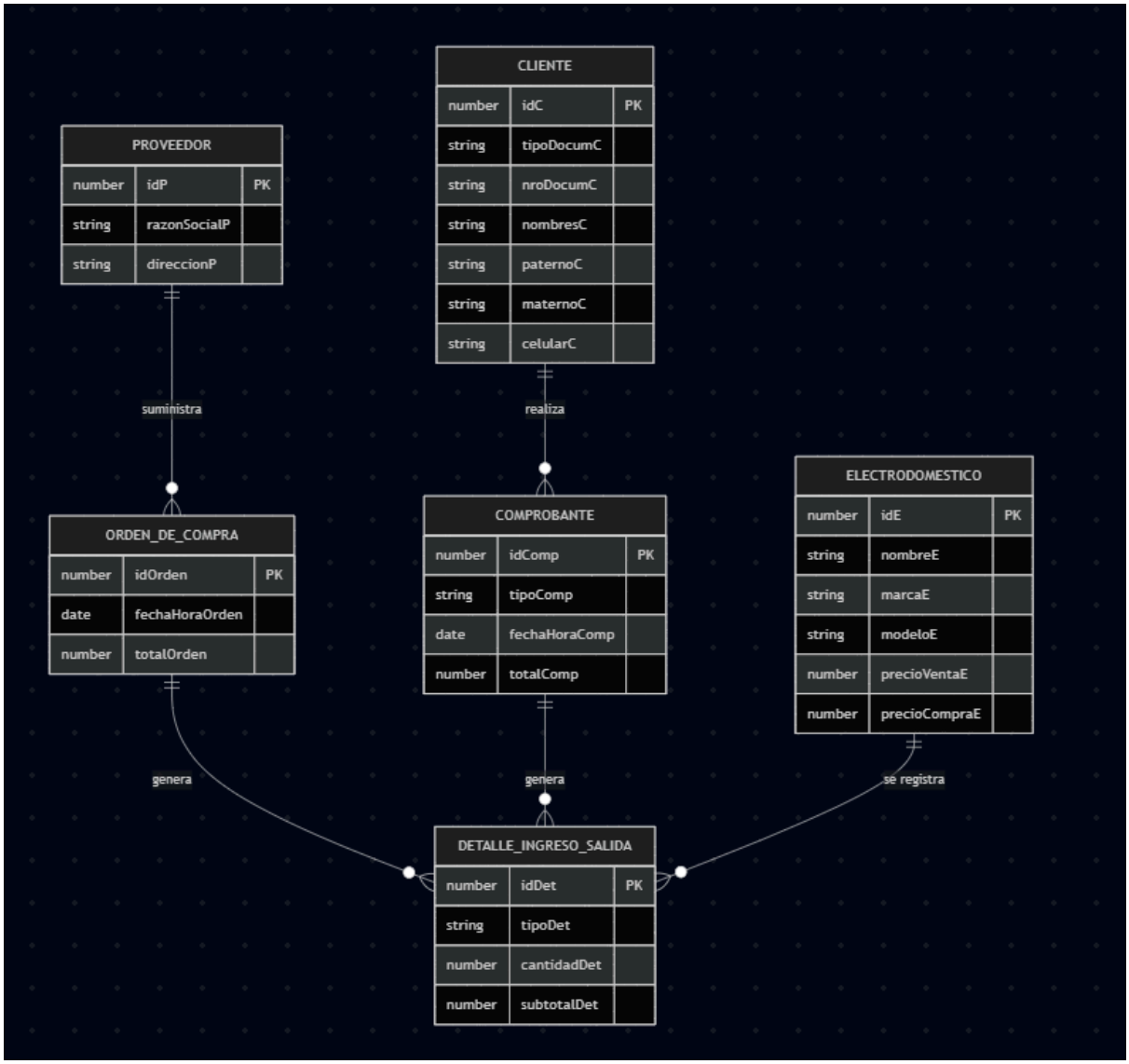
Código

```
-- LIMPIEZA INICIAL
BEGIN
  EXECUTE IMMEDIATE 'DROP TABLE ET_Detalle_Ingreso_Salida CASCADE
CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE ET_Electrodomestico CASCADE
CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE ET_Comprobante CASCADE
CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE ET_Orden_Compra CASCADE
CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE ET_Cliente CASCADE CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE ET_Proveedor CASCADE
CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

-- 1. Tabla Proveedor [cite: 116]
CREATE TABLE ET_Proveedor (
   idP          NUMBER(10) PRIMARY KEY, -- [cite: 117]
   razonSocialP  VARCHAR2(100),         -- [cite: 120]
   direccionP    VARCHAR2(150)          -- [cite: 121]
);
```

```
-- 2. Tabla Cliente [cite: 122]
CREATE TABLE ET_Cliente (
    idC          NUMBER(10) PRIMARY KEY, -- [cite: 123]
    tipoDocumC    VARCHAR2(20),          -- [cite: 124]
    nroDocumC     VARCHAR2(20) UNIQUE,    -- [cite: 130]
    nombresC      VARCHAR2(100),          -- [cite: 129]
    paternoC      VARCHAR2(100),          -- [cite: 132]
    maternoC      VARCHAR2(100),          -- [cite: 133]
    celularC      VARCHAR2(20)            -- [cite: 134]
);

-- 3. Tabla Orden de Compra [cite: 118]
CREATE TABLE ET_Orden_Compra (
    idOrden       NUMBER(10) PRIMARY KEY, -- [cite: 119]
    idP           NUMBER(10) NOT NULL,
    fechaHoraOrden DATE DEFAULT SYSDATE,  -- [cite: 125]
    totalOrden    NUMBER(12, 2),          -- [cite: 125]
    CONSTRAINT fk_et_prov_ord FOREIGN KEY (idP) REFERENCES
ET_Proveedor(idP)
);

-- 4. Tabla Comprobante [cite: 126]
```

```
CREATE TABLE ET_Comprobante (
    idComp        NUMBER(10) PRIMARY KEY, -- [cite: 127]
    idC           NUMBER(10) NOT NULL,
    tipoComp      VARCHAR2(20),           -- [cite: 128]
    fechaHoraComp DATE DEFAULT SYSDATE,   -- [cite: 131]
    totalComp     NUMBER(12, 2),          -- [cite: 131]
    CONSTRAINT fk_et_cli_comp FOREIGN KEY (idC) REFERENCES
ET_Cliente(idC)
);

-- 5. Tabla Electrodoméstico [cite: 142]
CREATE TABLE ET_Electrodomestico (
    idE           NUMBER(10) PRIMARY KEY, -- [cite: 143]
    nombreE       VARCHAR2(100),          -- [cite: 144]
    marcaE        VARCHAR2(50),           -- [cite: 145]
    modeloE       VARCHAR2(50),           -- [cite: 146]
    precioVentaE  NUMBER(10, 2),          -- [cite: 147]
    precioCompraE NUMBER(10, 2)           -- [cite: 147]
);

-- 6. Tabla Detalle de Ingreso/Salida [cite: 137]
CREATE TABLE ET_Detalle_Ingreso_Salida (
    idDet         NUMBER(10) PRIMARY KEY, -- [cite: 138]
```

```
idOrden       NUMBER(10),            -- Relación opcional con Orden
idComp        NUMBER(10),            -- Relación opcional con Comprobante
idE           NUMBER(10) NOT NULL,   -- Relación con Producto
tipoDet       VARCHAR2(20),          -- [cite: 139]
cantidadDet   NUMBER(10),            -- [cite: 141]
subtotalDet   NUMBER(10, 2),         -- [cite: 140]
CONSTRAINT fk_et_ord_det FOREIGN KEY (idOrden) REFERENCES
ET_Orden_Compra(idOrden),
CONSTRAINT fk_et_comp_det FOREIGN KEY (idComp) REFERENCES
ET_Comprobante(idComp),
CONSTRAINT fk_et_prod_det FOREIGN KEY (idE) REFERENCES
ET_Electrodomestico(idE)
);
```



6.- Sistema de registro de asistencia de las enfermeras a la clínica Mac Salud

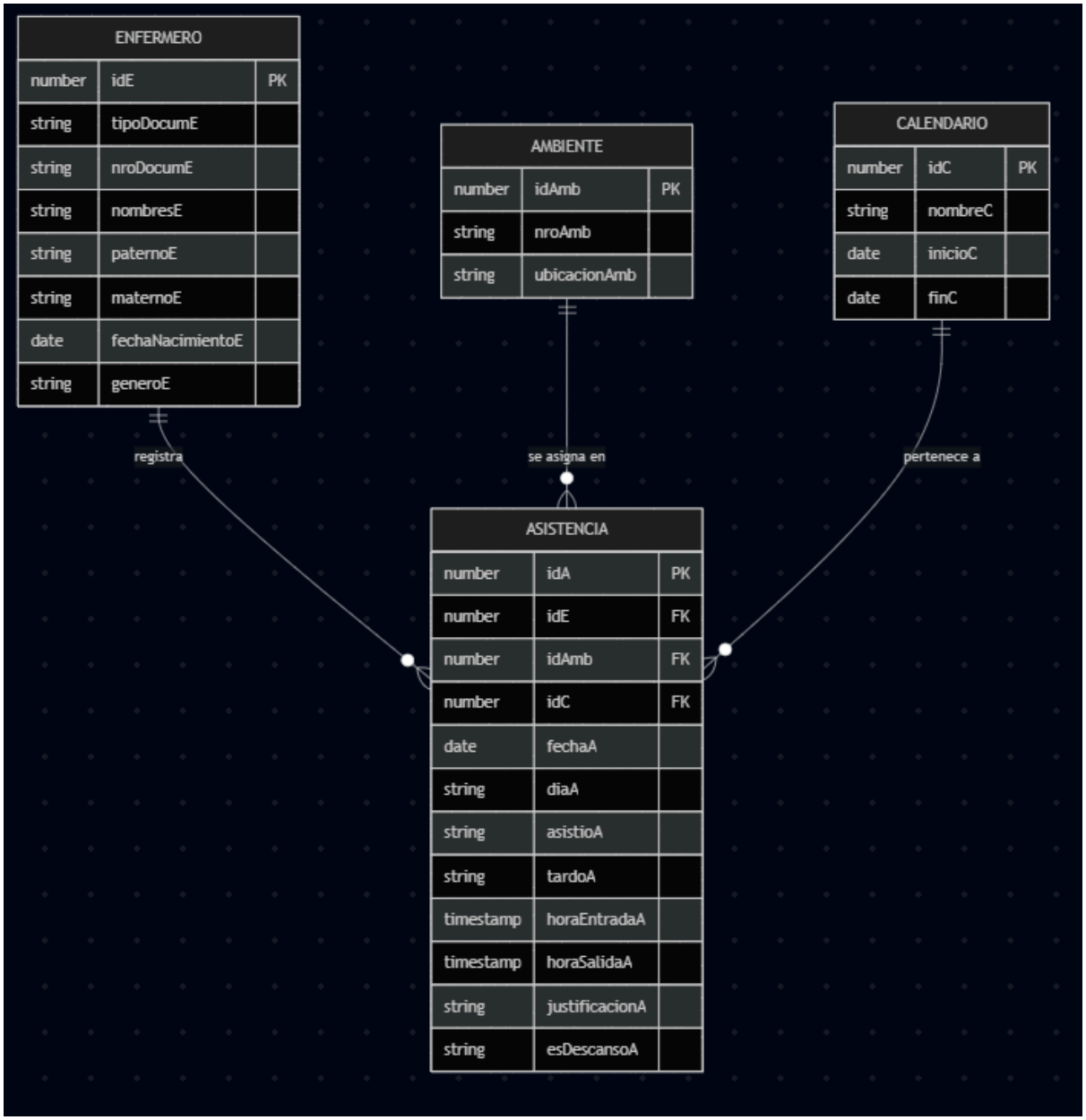
Código:

```
-- LIMPIEZA INICIAL
BEGIN
   EXECUTE IMMEDIATE 'DROP TABLE MS_Asistencia CASCADE
CONSTRAINTS';
```

```
        EXECUTE IMMEDIATE 'DROP TABLE MS_Enfermero CASCADE
CONSTRAINTS';
        EXECUTE IMMEDIATE 'DROP TABLE MS_Ambiente CASCADE
CONSTRAINTS';
        EXECUTE IMMEDIATE 'DROP TABLE MS_Calendario CASCADE
CONSTRAINTS';
    EXCEPTION WHEN OTHERS THEN NULL;
    END;
    /

    -- 1. Tabla Enfermero
    CREATE TABLE MS_Enfermero (
      idE              NUMBER(10) PRIMARY KEY,
      tipoDocumE       VARCHAR2(20),
      nroDocumE        VARCHAR2(20) UNIQUE,
      nombresE         VARCHAR2(100),
      paternoE         VARCHAR2(100),
      maternoE         VARCHAR2(100),
      fechaNacimientoE  DATE,
      generoE          VARCHAR2(10)
    );

    -- 2. Tabla Ambiente
```

```
CREATE TABLE MS_Ambiente (
    idAmb           NUMBER(10) PRIMARY KEY,
    nroAmb          VARCHAR2(10),
    ubicacionAmb    VARCHAR2(100)
);

-- 3. Tabla Calendario
CREATE TABLE MS_Calendario (
    idC             NUMBER(10) PRIMARY KEY,
    nombreC         VARCHAR2(50),
    inicioC         DATE,
    finC            DATE
);

-- 4. Tabla Asistencia
CREATE TABLE MS_Asistencia (
    idA             NUMBER(10) PRIMARY KEY,
    idE             NUMBER(10) NOT NULL,
    idAmb           NUMBER(10) NOT NULL,
    idC             NUMBER(10) NOT NULL,
    fechaA          DATE,
    diaA            VARCHAR2(20),
    asistioA        CHAR(1), -- 'S' o 'N'
```

```
        tardoA          CHAR(1), -- 'S' o 'N'
        horaEntradaA     TIMESTAMP,
        horaSalidaA      TIMESTAMP,
        justificacionA   VARCHAR2(255),
        esDescansoA      CHAR(1), -- 'S' o 'N'
        CONSTRAINT fk_ms_enf_asist FOREIGN KEY (idE) REFERENCES
MS_Enfermero(idE),
        CONSTRAINT fk_ms_amb_asist FOREIGN KEY (idAmb) REFERENCES
MS_Ambiente(idAmb),
        CONSTRAINT fk_ms_cal_asist FOREIGN KEY (idC) REFERENCES
MS_Calendario(idC)
    );
```



7.- Sistema de facturación electrónica de la pollería Los Ángeles

Código:

```
-- LIMPIEZA INICIAL
BEGIN
   EXECUTE IMMEDIATE 'DROP TABLE LA_Detalle CASCADE CONSTRAINTS';
```

```
    EXECUTE IMMEDIATE 'DROP TABLE LA_Comprobante CASCADE
CONSTRAINTS';
    EXECUTE IMMEDIATE 'DROP TABLE LA_Producto CASCADE CONSTRAINTS';
    EXECUTE IMMEDIATE 'DROP TABLE LA_Categoria CASCADE
CONSTRAINTS';
    EXECUTE IMMEDIATE 'DROP TABLE LA_Cliente CASCADE CONSTRAINTS';
  EXCEPTION WHEN OTHERS THEN NULL;
  END;
  /

  -- 1. Tabla Categoría
  CREATE TABLE LA_Categoria (
    idCat         NUMBER(10) PRIMARY KEY,
    nombreCat     VARCHAR2(100),
    descripcionCat VARCHAR2(255)
  );

  -- 2. Tabla Producto
  CREATE TABLE LA_Producto (
    idP             NUMBER(10) PRIMARY KEY,
    idCat           NUMBER(10) NOT NULL,
    nombreP         VARCHAR2(100),
    decripcionP     VARCHAR2(255),
```

```
    precioRefrencialP NUMBER(10, 2),
    CONSTRAINT fk_la_cat_prod FOREIGN KEY (idCat) REFERENCES
LA_Categoria(idCat)
);

-- 3. Tabla Cliente
CREATE TABLE LA_Cliente (
    idC          NUMBER(10) PRIMARY KEY,
    tipoDocumC   VARCHAR2(20),
    nroDocumC    VARCHAR2(20) UNIQUE,
    nombresC     VARCHAR2(100),
    paternoC     VARCHAR2(100),
    maternoC     VARCHAR2(100),
    celularC     VARCHAR2(20)
);

-- 4. Tabla Comprobante
CREATE TABLE LA_Comprobante (
    idComp        NUMBER(10) PRIMARY KEY,
    idC           NUMBER(10) NOT NULL,
    fechaHoraComp  DATE DEFAULT SYSDATE,
    totalComp      NUMBER(12, 2),
```

```
    CONSTRAINT fk_la_cli_comp FOREIGN KEY (idC) REFERENCES
LA_Cliente(idC)
);

-- 5. Tabla Detalle
CREATE TABLE LA_Detalle (
    idDet                 NUMBER(10) PRIMARY KEY,
    idComp                NUMBER(10) NOT NULL,
    idP                   NUMBER(10) NOT NULL,
    cantidadDet           NUMBER(10),
    precioRealUnitarioDet  NUMBER(10, 2),
    subtotalDet           NUMBER(10, 2),
    CONSTRAINT fk_la_comp_det FOREIGN KEY (idComp) REFERENCES
LA_Comprobante(idComp),
    CONSTRAINT fk_la_prod_det FOREIGN KEY (idP) REFERENCES
LA_Producto(idP)
);
```

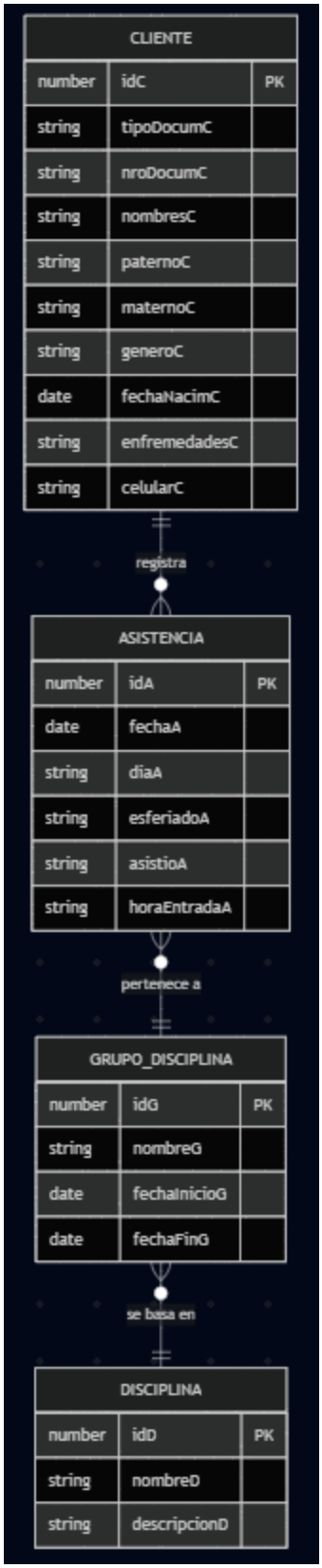
8.- Registro de asistencia de clientes del gimnasio ‘Titanium

Código:

```
-- LIMPIEZA INICIAL
BEGIN
  EXECUTE IMMEDIATE 'DROP TABLE AD_Asistencia CASCADE
CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE AD_GrupoDisciplina CASCADE
CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE AD_Disciplina CASCADE
CONSTRAINTS';
  EXECUTE IMMEDIATE 'DROP TABLE AD_Cliente CASCADE CONSTRAINTS';
EXCEPTION WHEN OTHERS THEN NULL;
END;
/

-- 1. Tabla Disciplina
CREATE TABLE AD_Disciplina (
  idD          NUMBER(10) PRIMARY KEY,
  nombreD      VARCHAR2(100),
  descripcionD VARCHAR2(255)
);

-- 2. Tabla GrupoDisciplina
CREATE TABLE AD_GrupoDisciplina (
  idG          NUMBER(10) PRIMARY KEY,
```

```
    idD           NUMBER(10) NOT NULL,
    nombreG       VARCHAR2(100),
    fechaInicioG  DATE,
    fechaFinG     DATE,
    CONSTRAINT fk_ad_disc_grp FOREIGN KEY (idD) REFERENCES
AD_Disciplina(idD)
);

-- 3. Tabla Cliente
CREATE TABLE AD_Cliente (
    idC           NUMBER(10) PRIMARY KEY,
    tipoDocumC    VARCHAR2(20),
    nroDocumC     VARCHAR2(20) UNIQUE,
    nombresC      VARCHAR2(100),
    paternoC      VARCHAR2(100),
    maternoC      VARCHAR2(100),
    generoC       VARCHAR2(10),
    fechaNacimC   DATE,
    enfremedadesC VARCHAR2(255), -- Tal cual aparece en la imagen
    celularC      VARCHAR2(20)
);

-- 4. Tabla Asistencia
```

```
CREATE TABLE AD_Asistencia (
    idA            NUMBER(10) PRIMARY KEY,
    idC            NUMBER(10) NOT NULL,
    idG            NUMBER(10) NOT NULL,
    fechaA         DATE DEFAULT SYSDATE,
    diaA           VARCHAR2(20),
    esferiadoA     CHAR(1),
    asistioA       CHAR(1),
    horaEntradaA   VARCHAR2(20),
    CONSTRAINT fk_ad_cli_asist FOREIGN KEY (idC) REFERENCES
AD_Cliente(idC),
    CONSTRAINT fk_ad_grp_asist FOREIGN KEY (idG) REFERENCES
AD_GrupoDisciplina(idG)
);
```


CLIENTE
number idC PK
string tipoDocumC
string nroDocumC
string nombresC
string paternoC
string maternoC
string generoC
date fechaNacimC
string enfremedadesC
string celularC
registra
ASISTENCIA
number idA PK
date fechaA
string diaA
string esferiadoA
string asistioA
string horaEntradaA
pertenece a
GRUPO_DISCIPLINA
number idG PK
string nombreG
date fechaInicioG
date fechaFinG
se basa en
DISCIPLINA
number idD PK
string nombreD
string descripcionD

SQL SERVER

2. Sistema de registro de transacciones bancarias a nivel nacional - Perú de la empresa Super Giros.

```
use master
go

if DB_ID('BDBanco') is not null
	drop database BDBanco
go
create database BDBanco
go

use BDBanco
go

if OBJECT_ID('Banco') is not null
	drop table Banco
go
```

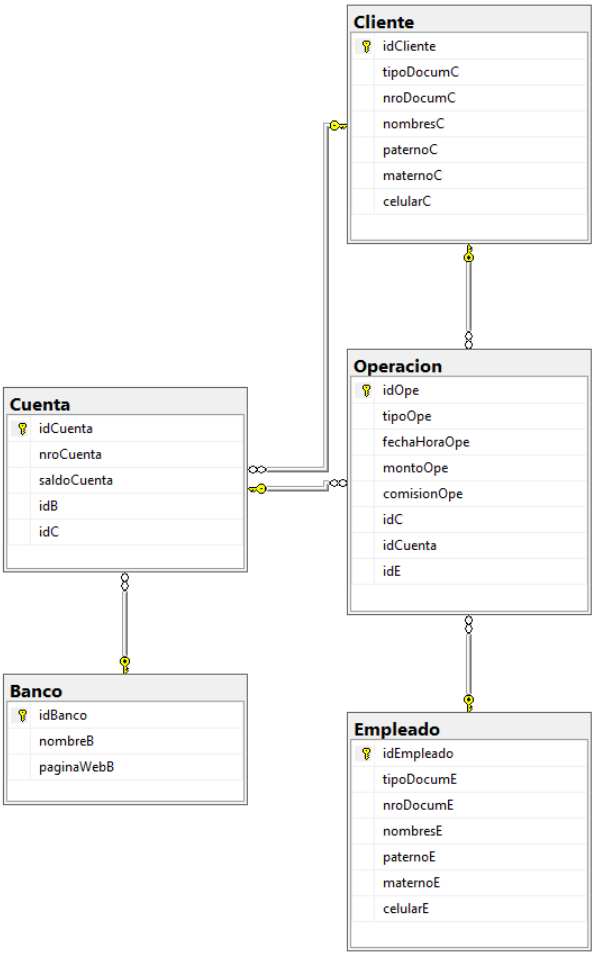
```
create table Banco
(
	idBanco int primary key identity(1,1),
	nombreB varchar (100) not null,
	paginaWebB varchar(150) null
)
go

if OBJECT_ID('Cliente') is not null
	drop table Cliente
go
create table Cliente
(
	idCliente int primary key identity (1,1),
	tipoDocumC varchar (20)not null,
	nroDocumC varchar (20)not null,
	nombresC varchar (100)not null,
	paternoC varchar (100)not null,
	maternoC varchar (100)not null,
	celularC char (9) not null
)
go
```

```
alter table Cliente
add constraint UQ_Cliente_nroDocumC unique (nroDocumC)
go

if OBJECT_ID('Empleado') is not null
	drop table Empleado
go
create table Empleado
(
	idEmpleado int primary key identity (1,1),
	tipoDocumE varchar (20) not null,
	nroDocumE varchar (20) not null,
	nombresE varchar (100) not null,
	paternoE varchar (100) not null,
	maternoE varchar (100) not null,
	celularE char(9)
)
go
alter table Empleado
add constraint UQ_Empleado_nroDocumE unique (nroDocumE)
go

if OBJECT_ID('Cuenta') is not null
```

```
        drop table Cuenta
go
create table Cuenta
(
        idCuenta int primary key identity(1,1),
        nroCuenta varchar(20) not null,
        saldoCuenta decimal(12,2) not null,
        idB int not null,
        idC int not null,

        constraint UQ_Cuenta_nroCuenta unique (nroCuenta),
        constraint CK_Cuenta_saldoCuenta check (saldoCuenta >= 0),

        constraint FK_Banco_Cuenta
        foreign key (idB)
        references Banco(idBanco),

        constraint FK_Cliente_Cuenta
        foreign key (idC)
        references Cliente(idCliente)
)
go
```

```
if OBJECT_ID('Operacion') is not null
	drop table Operacion
go

create table Operacion
(
	idOpe int primary key identity(1,1),
	tipoOpe varchar(30) not null,
	fechaHoraOpe datetime not null,
	montoOpe decimal(12,2) not null,
	comisionOpe decimal(12,2) not null,
	idC int not null,
	idCuenta int not null,
	idE int not null,

	constraint CK_Operacion_montoOpe
	check (montoOpe > 0),

	constraint CK_Operacion_comisionOpe
	check (comisionOpe >= 0),

	constraint FK_Cliente_Operacion
	foreign key (idC)
```

```
        references Cliente(idCliente),

        constraint FK_Cuenta_Operacion
        foreign key (idCuenta)
        references Cuenta(idCuenta),

        constraint FK_Empleado_Operacion
        foreign key (idE)
        references Empleado(idEmpleado)
)
go
```



3. Sistema de registro de asistencia del personal del INPE.

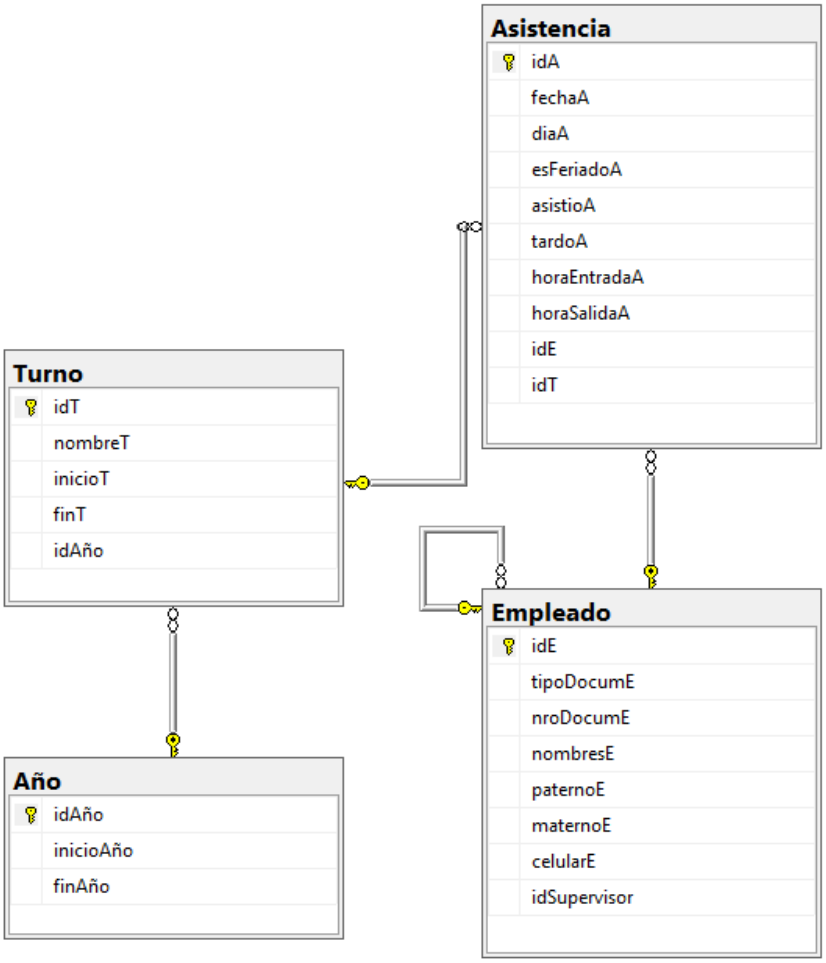
```
use master
go

if DB_ID('BDAsistencia') is not null
	drop database BDAsistencia
go
create database BDAsistencia
go

use BDAsistencia
go

if OBJECT_ID('Asistencia') is not null
	drop table Asistencia
go

if OBJECT_ID('Turno') is not null
	drop table Turno
go

if OBJECT_ID('Empleado') is not null
```

```
	drop table Empleado
go

if OBJECT_ID('[Año]') is not null
	drop table [Año]
go

create table [Año]
(
	[idAño] int primary key identity(1,1),
	[inicioAño] date not null,
	[finAño] date not null,

	constraint CK_Año_Fechas
	check ([inicioAño] <= [finAño])
)
go

create table Empleado
(
	idE int primary key identity(1,1),
	tipoDocumE varchar(20) not null,
	nroDocumE varchar(20) not null,
```

```
	nombresE varchar(100) not null,
	paternoE varchar(100) not null,
	maternoE varchar(100) not null,
	celularE char(9) not null,
	idSupervisor int null,

	constraint UQ_Empleado_nroDocumE unique (nroDocumE),

	constraint FK_Empleado_Supervisor
	foreign key (idSupervisor)
	references Empleado(idE)
)
go

create table Turno
(
	idT int primary key identity(1,1),
	nombreT varchar(50) not null,
	inicioT time not null,
	finT time not null,
	[idAño] int not null,

	constraint FK_Año_Turno
```

```
	foreign key ([idAño])
	references [Año]([idAño])
)
go

create table Asistencia
(
	idA int primary key identity(1,1),
	fechaA date not null,
	diaA varchar(20) not null,
	esFeriadoA bit not null,
	asistioA bit not null,
	tardoA bit not null,
	horaEntradaA time null,
	horaSalidaA time null,
	idE int not null,
	idT int not null,

	constraint FK_Empleado_Asistencia
	foreign key (idE)
	references Empleado(idE),

	constraint FK_Turno_Asistencia
```

```
        foreign key (idT)
        references Turno(idT),

        constraint CK_Asistencia_Horas
        check (
            (horaEntradaA is null and horaSalidaA is null)
            or
            (horaEntradaA is not null and horaSalidaA is not null and horaEntradaA
<= horaSalidaA)
        )
    )
    go
```



4. Sistema de registro de venta de platillos y bebidas en la pollería "Don Gallino"

```
use master
go

if DB_ID('BDVentas') is not null
        drop database BDVentas
go
create database BDVentas
```

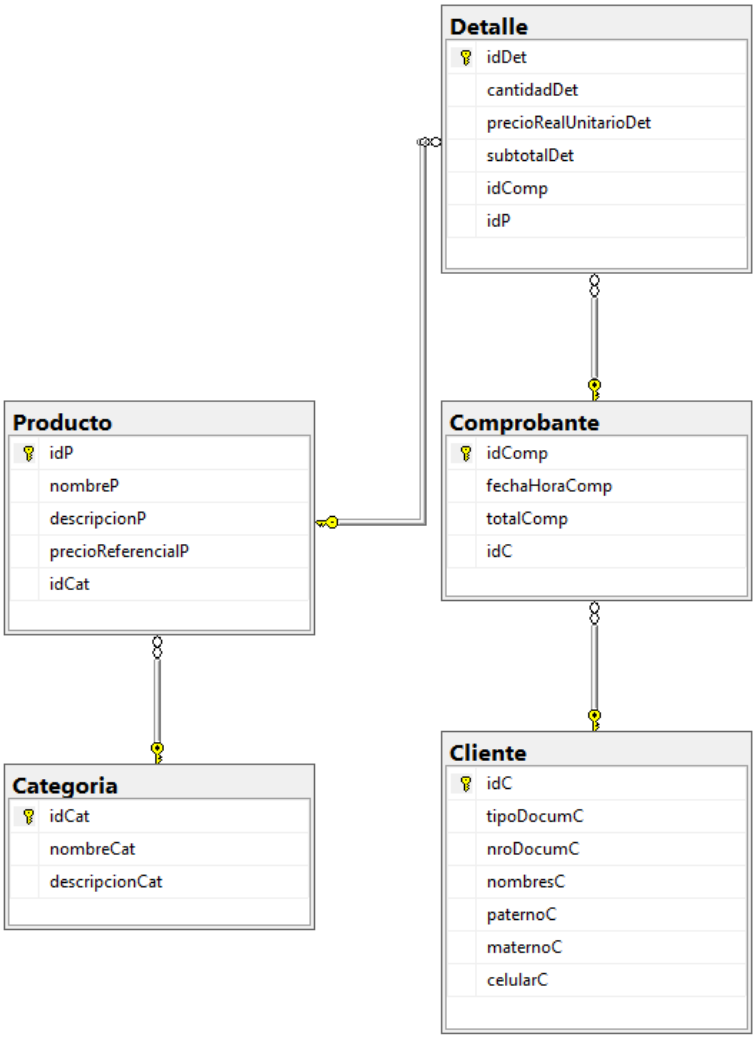
```
go

use BDVentas
go

if OBJECT_ID('Detalle') is not null
	drop table Detalle
go

if OBJECT_ID('Producto') is not null
	drop table Producto
go

if OBJECT_ID('Categoria') is not null
	drop table Categoria
go

if OBJECT_ID('Comprobante') is not null
	drop table Comprobante
go

if OBJECT_ID('Cliente') is not null
	drop table Cliente
```

```
go

create table Cliente
(
	idC int primary key identity(1,1),
	tipoDocumC varchar(20) not null,
	nroDocumC varchar(20) not null,
	nombresC varchar(100) not null,
	paternoC varchar(100) not null,
	maternoC varchar(100) not null,
	celularC char(9) not null
)
go

alter table Cliente
add constraint UQ_Cliente_nroDocumC unique (nroDocumC)
go

create table Categoria
(
	idCat int primary key identity(1,1),
	nombreCat varchar(100) not null,
	descripcionCat varchar(200) not null
```

```
)
go

create table Producto
(
	idP int primary key identity(1,1),
	nombreP varchar(100) not null,
	descripcionP varchar(200) not null,
	precioReferencialP decimal(10,2) not null,
	idCat int not null,

	constraint CK_Producto_precioReferencialP
	check (precioReferencialP >= 0),

	constraint FK_Categoria_Producto
	foreign key (idCat)
	references Categoria(idCat)
)
go

create table Comprobante
(
	idComp int primary key identity(1,1),
```

```
	fechaHoraComp datetime not null,
	totalComp decimal(12,2) not null,
	idC int not null,

	constraint CK_Comprobante_totalComp
	check (totalComp >= 0),

	constraint FK_Cliente_Comprobante
	foreign key (idC)
	references Cliente(idC)
)
go

create table Detalle
(
	idDet int primary key identity(1,1),
	cantidadDet int not null,
	precioRealUnitarioDet decimal(10,2) not null,
	subtotalDet decimal(12,2) not null,
	idComp int not null,
	idP int not null,

	constraint CK_Detalle_cantidadDet
```

```
        check (cantidadDet > 0),

        constraint CK_Detalle_precios
        check (precioRealUnitarioDet >= 0 and subtotalDet >= 0),

        constraint FK_Comprobante_Detalle
        foreign key (idComp)
        references Comprobante(idComp),

        constraint FK_Producto_Detalle
        foreign key (idP)
        references Producto(idP)
)
go
```



5. Sistema de registro de entrada y salida de electrodomésticos "Electro Track"

```
use master
go

if DB_ID('BDInventario') is not null
        drop database BDInventario
go
create database BDInventario
```

```
go

use BDInventario
go

if OBJECT_ID('detalleIngresoSalida') is not null
	drop table detalleIngresoSalida
go

if OBJECT_ID('Electrodomestico') is not null
	drop table Electrodomestico
go

if OBJECT_ID('Comprobante') is not null
	drop table Comprobante
go

if OBJECT_ID('OrdenDeCompra') is not null
	drop table OrdenDeCompra
go

if OBJECT_ID('Cliente') is not null
	drop table Cliente
```

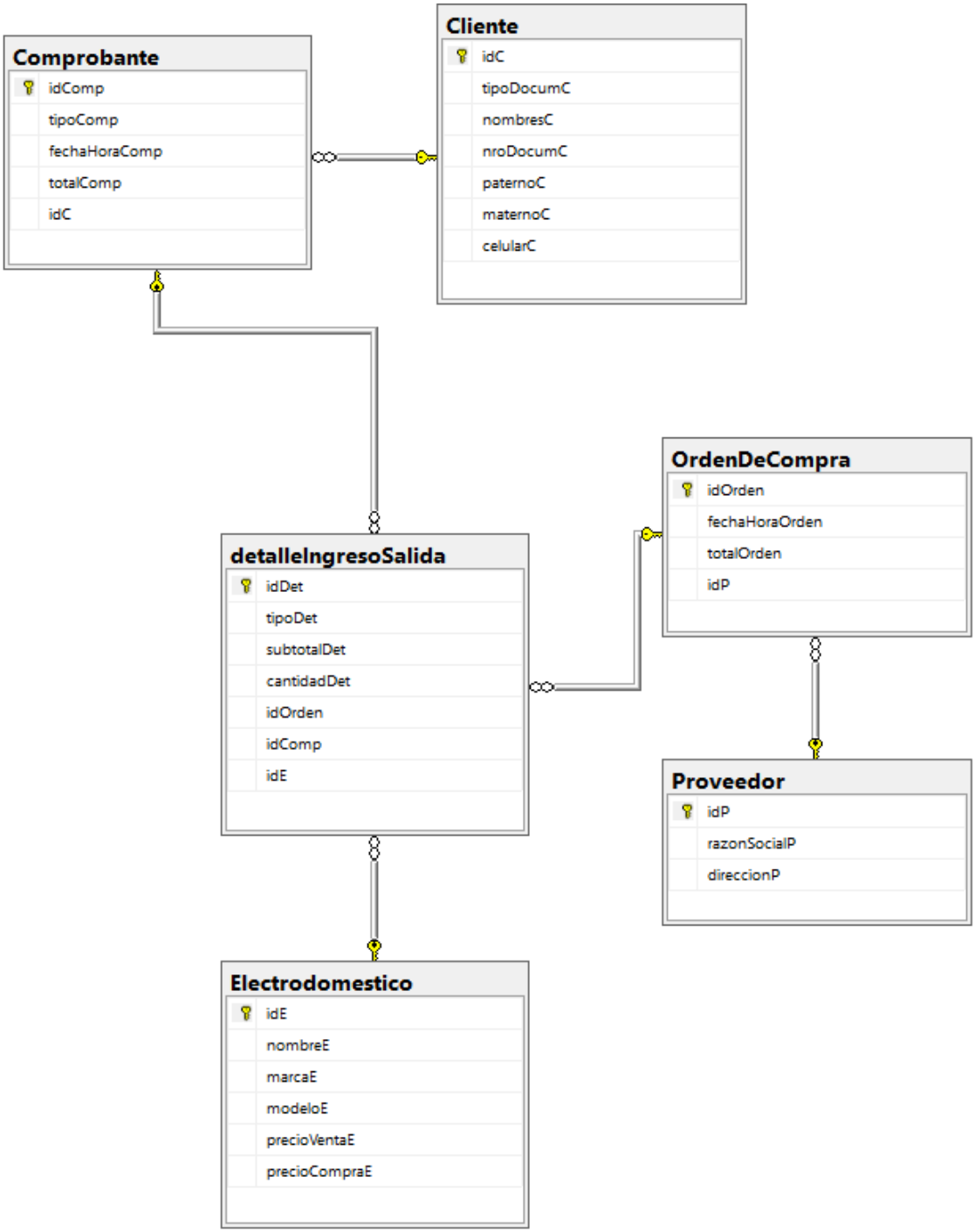
```
go

if OBJECT_ID('Proveedor') is not null
	drop table Proveedor
go


create table Proveedor
(
	idP int primary key identity(1,1),
	razonSocialP varchar(150) not null,
	direccionP varchar(200) not null
)
go


create table Cliente
(
	idC int primary key identity(1,1),
	tipoDocumC varchar(20) not null,
	nombresC varchar(100) not null,
	nroDocumC varchar(20) not null,
	paternoC varchar(100) not null,
```

```
	maternoC varchar(100) not null,
	celularC char(9) not null
)
go

alter table Cliente
add constraint UQ_Cliente_nroDocumC unique (nroDocumC)
go



create table Electrodomestico
(
	idE int primary key identity(1,1),
	nombreE varchar(100) not null,
	marcaE varchar(100) not null,
	modeloE varchar(100) not null,
	precioVentaE decimal(10,2) not null,
	precioCompraE decimal(10,2) not null,

	constraint CK_Electrodomestico_precios
	check (precioVentaE >= 0 and precioCompraE >= 0)
)
go
```

```
create table OrdenDeCompra
(
	idOrden int primary key identity(1,1),
	fechaHoraOrden datetime not null,
	totalOrden decimal(12,2) not null,
	idP int not null,

	constraint CK_Orden_total
	check (totalOrden >= 0),

	constraint FK_Proveedor_Orden
	foreign key (idP)
	references Proveedor(idP)
)
go


create table Comprobante
(
	idComp int primary key identity(1,1),
	tipoComp varchar(50) not null,
```

```
	fechaHoraComp datetime not null,
	totalComp decimal(12,2) not null,
	idC int not null,

	constraint CK_Comprobante_total
	check (totalComp >= 0),

	constraint FK_Cliente_Comprobante
	foreign key (idC)
	references Cliente(idC)
)
go


create table detalleIngresoSalida
(
	idDet int primary key identity(1,1),
	tipoDet varchar(30) not null,
	subtotalDet decimal(12,2) not null,
	cantidadDet int not null,
	idOrden int null,
	idComp int null,
	idE int not null,
```

```
        constraint CK_detalle_cantidad
        check (cantidadDet > 0),

        constraint CK_detalle_subtotal
        check (subtotalDet >= 0),

        constraint FK_Orden_detalle
        foreign key (idOrden)
        references OrdenDeCompra(idOrden),

        constraint FK_Comprobante_detalle
        foreign key (idComp)
        references Comprobante(idComp),

        constraint FK_Electrodomestico_detalle
        foreign key (idE)
        references Electrodomestico(idE)
)
go
```


Comprobante
idComp
tipoComp
fechaHoraComp
totalComp
idC
Cliente
idC
tipoDocumC
nombresC
nroDocumC
paternoC
maternoC
celularC
OrdenDeCompra
idOrden
fechaHoraOrden
totalOrden
idP
detalleIngresoSalida
idDet
tipoDet
subtotalDet
cantidadDet
idOrden
idComp
idE
Proveedor
idP
razonSocialP
direccionP
Electrodomestico
idE
nombreE
marcaE
modeloE
precioVentaE
precioCompraE

6. Sistema de registro de asistencia de las enfermeras a la clínica Mac Salud

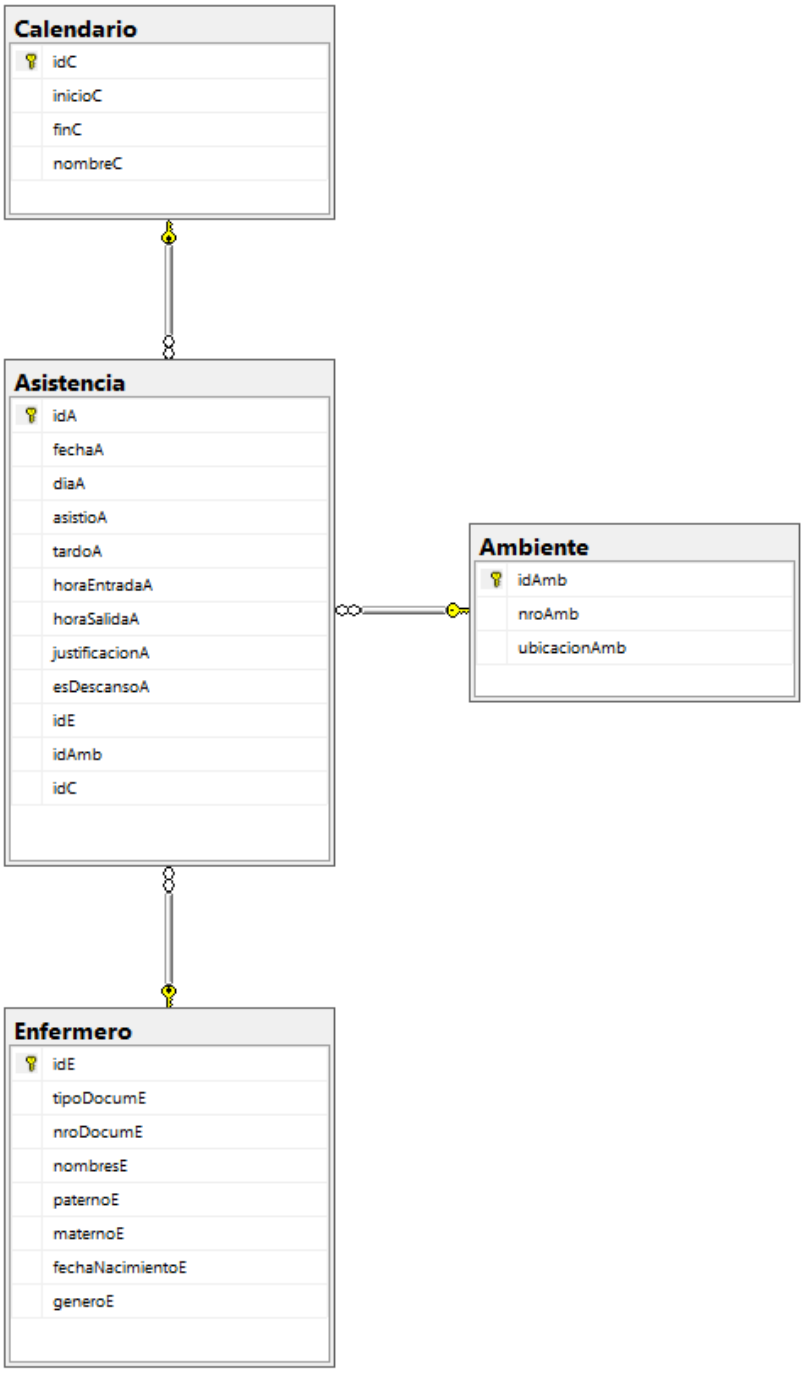
```
use master
go

if DB_ID('BDHospital') is not null
	drop database BDHospital
go
create database BDHospital
go

use BDHospital
go

if OBJECT_ID('Asistencia') is not null
	drop table Asistencia
go
```

```
if OBJECT_ID('Calendario') is not null
	drop table Calendario
go

if OBJECT_ID('Ambiente') is not null
	drop table Ambiente
go

if OBJECT_ID('Enfermero') is not null
	drop table Enfermero
go


create table Enfermero
(
	idE int primary key identity(1,1),
	tipoDocumE varchar(20) not null,
	nroDocumE varchar(20) not null,
	nombresE varchar(100) not null,
	paternoE varchar(100) not null,
	maternoE varchar(100) not null,
	fechaNacimientoE date not null,
```

```
	generoE varchar(10) not null
)
go

alter table Enfermero
add constraint UQ_Enfermero_nroDocumE unique (nroDocumE)
go


create table Ambiente
(
	idAmb int primary key identity(1,1),
	nroAmb varchar(20) not null,
	ubicacionAmb varchar(150) not null
)
go


create table Calendario
(
	idC int primary key identity(1,1),
	inicioC date not null,
	finC date not null,
```

```
	nombreC varchar(100) not null,


	constraint CK_Calendario_fechas
	check (inicioC <= finC)
)
go



create table Asistencia
(
	idA int primary key identity(1,1),
	fechaA date not null,
	diaA varchar(20) not null,
	asistioA bit not null,
	tardoA bit not null,
	horaEntradaA time null,
	horaSalidaA time null,
	justificacionA varchar(200) null,
	esDescansoA bit not null,
	idE int not null,
	idAmb int not null,
	idC int not null,
```

```
        constraint FK_Enfermero_Asistencia
        foreign key (idE)
        references Enfermero(idE),

        constraint FK_Ambiente_Asistencia
        foreign key (idAmb)
        references Ambiente(idAmb),

        constraint FK_Calendario_Asistencia
        foreign key (idC)
        references Calendario(idC),

        constraint CK_Asistencia_horas
        check (
            (horaEntradaA is null and horaSalidaA is null)
            or
            (horaEntradaA is not null and horaSalidaA is not null and horaEntradaA
<= horaSalidaA)
        )
    )
    go
```



7. Sistema de facturación electrónica de la pollería Los Ángeles.

use master

go

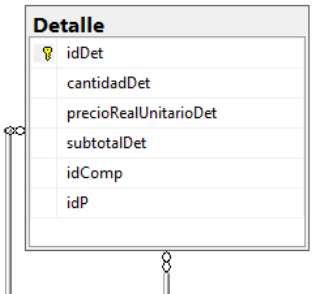
```
if DB_ID('BDVentas') is not null
begin
	alter database BDVentas set single_user with rollback immediate
	drop database BDVentas
end
go
create database BDVentas
go

use BDVentas
go

if OBJECT_ID('Detalle') is not null
	drop table Detalle
go

if OBJECT_ID('Producto') is not null
	drop table Producto
go

if OBJECT_ID('Categoria') is not null
	drop table Categoria
```

```
go

if OBJECT_ID('Comprobante') is not null
	drop table Comprobante
go

if OBJECT_ID('Cliente') is not null
	drop table Cliente
go

create table Cliente
(
	idC int primary key identity(1,1),
	tipoDocumC varchar(20) not null,
	nroDocumC varchar(20) not null,
	nombresC varchar(100) not null,
	paternoC varchar(100) not null,
	maternoC varchar(100) not null,
	celularC char(9) not null
)
go

alter table Cliente
```

```
add constraint UQ_Cliente_nroDocumC unique (nroDocumC)
go

create table Categoria
(
	idCat int primary key identity(1,1),
	nombreCat varchar(100) not null,
	descripcionCat varchar(200) not null
)
go

create table Producto
(
	idP int primary key identity(1,1),
	nombreP varchar(100) not null,
	decripcionP varchar(200) not null,
	precioReferencialP decimal(10,2) not null,
	idCat int not null,

	constraint CK_Producto_precioReferencialP
	check (precioReferencialP >= 0),

	constraint FK_Categoria_Producto
```

```
        foreign key (idCat)
        references Categoria(idCat)
)
go

create table Comprobante
(
        idComp int primary key identity(1,1),
        fechaHoraComp datetime not null,
        totalComp decimal(12,2) not null,
        idC int not null,

        constraint CK_Comprobante_totalComp
        check (totalComp >= 0),

        constraint FK_Cliente_Comprobante
        foreign key (idC)
        references Cliente(idC)
)
go

create table Detalle
(
```

```
        idDet int primary key identity(1,1),
        cantidadDet int not null,
        precioRealUnitarioDet decimal(10,2) not null,
        subtotalDet decimal(12,2) not null,
        idComp int not null,
        idP int not null,

        constraint CK_Detalle_cantidadDet
        check (cantidadDet > 0),

        constraint CK_Detalle_precios
        check (precioRealUnitarioDet >= 0 and subtotalDet >= 0),

        constraint FK_Comprobante_Detalle
        foreign key (idComp)
        references Comprobante(idComp),

        constraint FK_Producto_Detalle
        foreign key (idP)
        references Producto(idP)
)
go
```



8. Registro de asistencia de clientes del gimnasio ‘Titanium’

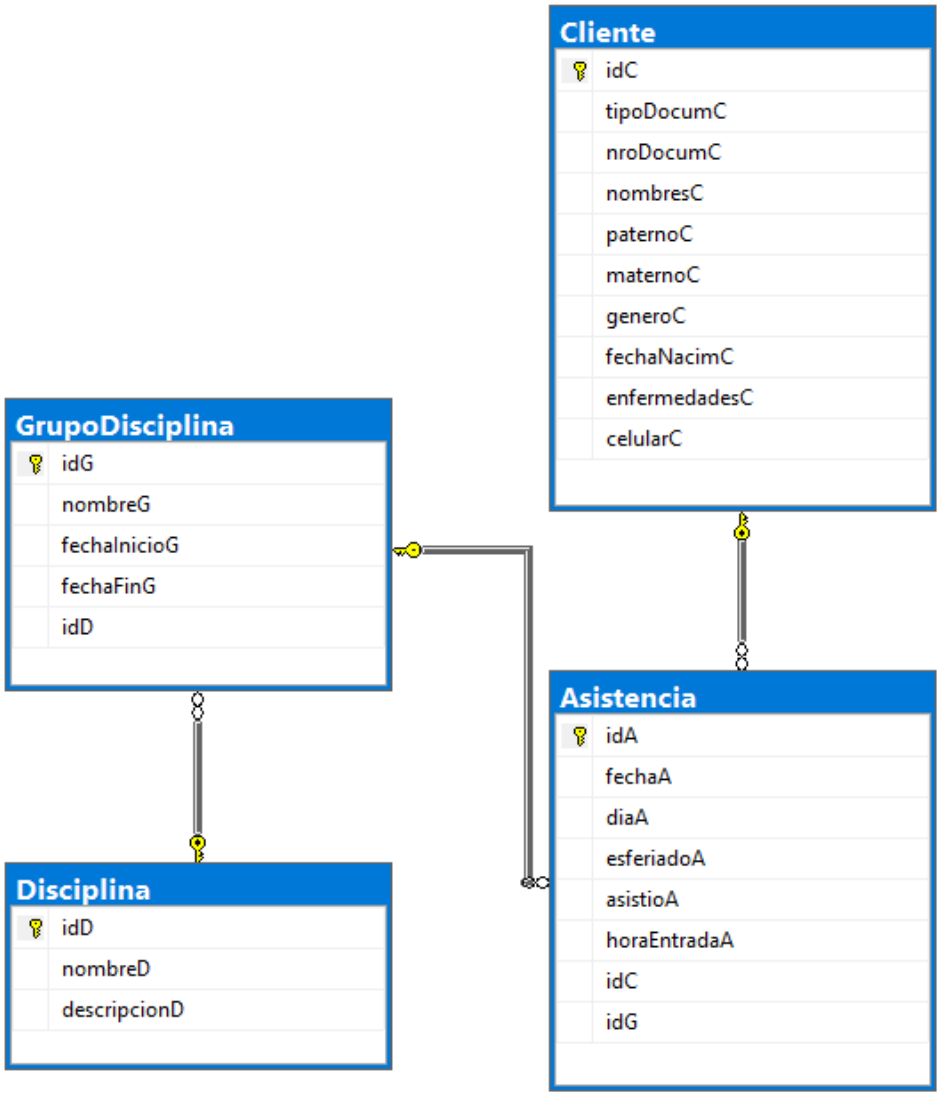
```
use master
go

if DB_ID('BDDisciplinas') is not null
begin
	alter database BDDisciplinas set single_user with rollback immediate
	drop database BDDisciplinas
end
go
create database BDDisciplinas
go

use BDDisciplinas
go

if OBJECT_ID('Asistencia') is not null
	drop table Asistencia
go

if OBJECT_ID('GrupoDisciplina') is not null
	drop table GrupoDisciplina
go
```

```
if OBJECT_ID('Disciplina') is not null
	drop table Disciplina
go

if OBJECT_ID('Cliente') is not null
	drop table Cliente
go



create table Cliente
(
	idC int primary key identity(1,1),
	tipoDocumC varchar(20) not null,
	nroDocumC varchar(20) not null,
	nombresC varchar(100) not null,
	paternoC varchar(100) not null,
	maternoC varchar(100) not null,
	generoC varchar(10) not null,
	fechaNacimC date not null,
	enfermedadesC varchar(200) null,
	celularC char(9) not null
)
```

```
go

alter table Cliente
add constraint UQ_Cliente_nroDocumC unique (nroDocumC)
go


create table Disciplina
(
	idD int primary key identity(1,1),
	nombreD varchar(100) not null,
	descripcionD varchar(200) not null
)
go


create table GrupoDisciplina
(
	idG int primary key identity(1,1),
	nombreG varchar(100) not null,
	fechaInicioG date not null,
	fechaFinG date not null,
	idD int not null,
```

```
	constraint CK_GrupoDisciplina_fechas
	check (fechaInicioG <= fechaFinG),

	constraint FK_Disciplina_GrupoDisciplina
	foreign key (idD)
	references Disciplina(idD)
)
go


create table Asistencia
(
	idA int primary key identity(1,1),
	fechaA date not null,
	diaA varchar(20) not null,
	esferiadoA bit not null,
	asistioA bit not null,
	horaEntradaA time null,
	idC int not null,
	idG int not null,

	constraint FK_Cliente_Asistencia
```

```
        foreign key (idC)
        references Cliente(idC),

        constraint FK_GrupoDisciplina_Asistencia
        foreign key (idG)
        references GrupoDisciplina(idG)
)
go
```


Cliente
idC
tipoDocumC
nroDocumC
nombresC
paternoC
maternoC
generoC
fechaNacimC
enfermedadesC
celularC
GrupoDisciplina
idG
nombreG
fechaInicioG
fechaFinG
idD
Asistencia
idA
fechaA
diaA
esferiadoA
asistioA
horaEntradaA
idC
idG
Disciplina
idD
nombreD
descripcionD

MYSQL

2. Sistema de registro de transacciones bancarias a nivel nacional - Perú de la empresa Super Giros.

CODIGO:

```
CREATE TABLE Banco (
    idBanco INT AUTO_INCREMENT PRIMARY KEY,
    nombreBanco VARCHAR(100)
);

CREATE TABLE Cliente (
    nroDocumC VARCHAR(15) PRIMARY KEY,
    tipoDocumC VARCHAR(20),
    nombresC VARCHAR(100),
    paternoC VARCHAR(100),
    maternoC VARCHAR(100),
```

```
    celularC VARCHAR(15)
);

CREATE TABLE Cuenta (
    idCuenta INT AUTO_INCREMENT PRIMARY KEY,
    nroCuenta VARCHAR(30),
    saldoCuenta DECIMAL(10,2),
    idBanco INT,
    nroDocumC VARCHAR(15),
    FOREIGN KEY (idBanco) REFERENCES Banco(idBanco),
    FOREIGN KEY (nroDocumC) REFERENCES Cliente(nroDocumC)
);

CREATE TABLE Empleado (
    nroDocumE VARCHAR(15) PRIMARY KEY,
    tipoDocumE VARCHAR(20),
    nombresE VARCHAR(100),
    paternoE VARCHAR(100),
    maternoE VARCHAR(100)
);

CREATE TABLE Operacion (
    idOperacion INT AUTO_INCREMENT PRIMARY KEY,
```

```
    tipoOperacion VARCHAR(50),
    montoOperacion DECIMAL(10,2),
    comisionOp DECIMAL(10,2),
    idCuenta INT,
    nroDocumE VARCHAR(15),
    FOREIGN KEY (idCuenta) REFERENCES Cuenta(idCuenta),
    FOREIGN KEY (nroDocumE) REFERENCES Empleado(nroDocumE)
);
```

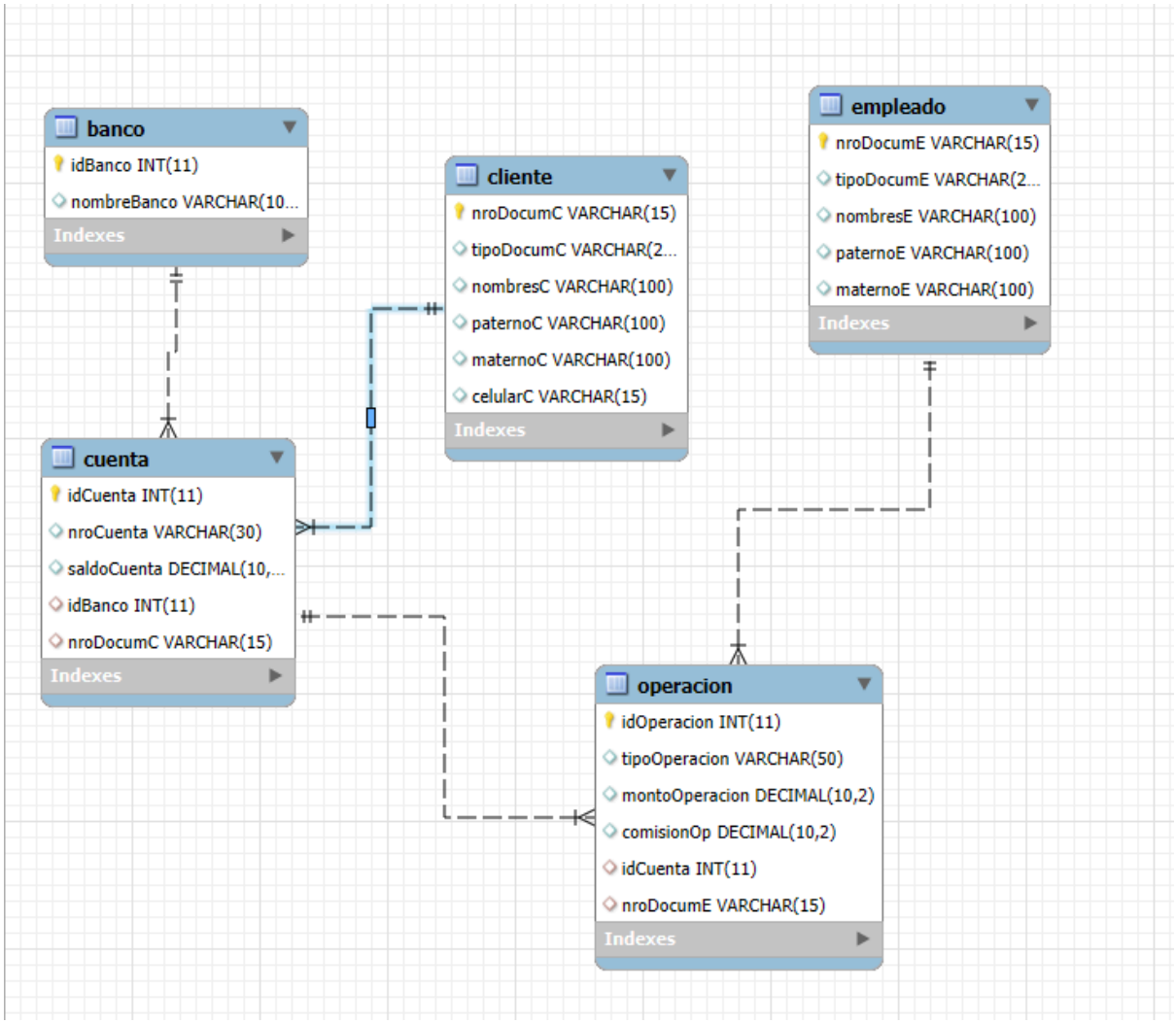
```sql
CREATE TABLE Cliente (
    nroDocumC VARCHAR(15) PRIMARY KEY,
    tipoDocumC VARCHAR(20),
    nombresC VARCHAR(100),
    paternoC VARCHAR(100),
    maternoC VARCHAR(100),
    celularC VARCHAR(15)
);

CREATE TABLE Cuenta (
    idCuenta INT AUTO_INCREMENT PRIMARY KEY,
    nroCuenta VARCHAR(30),
    saldoCuenta DECIMAL(10,2),
    idBanco INT,
    nroDocumC VARCHAR(15),
    FOREIGN KEY (idBanco) REFERENCES Banco(idBanco),
    FOREIGN KEY (nroDocumC) REFERENCES Cliente(nroDocumC)
);

CREATE TABLE Empleado (
    nroDocumE VARCHAR(15) PRIMARY KEY,
    tipoDocumE VARCHAR(20),
    nombresE VARCHAR(100),
    paternoE VARCHAR(100),
    maternoE VARCHAR(100)
);

CREATE TABLE Operacion (
    idOperacion INT AUTO_INCREMENT PRIMARY KEY,
    tipoOperacion VARCHAR(50),
    montoOperacion DECIMAL(10,2),
    comisionOp DECIMAL(10,2),
    idCuenta INT,
    nroDocumE VARCHAR(15),
    FOREIGN KEY (idCuenta) REFERENCES Cuenta(idCuenta),
    FOREIGN KEY (nroDocumE) REFERENCES Empleado(nroDocumE)
);
```

Output

Action Output

#	Time	Action	Message
17	11:11:25	DROP TABLE IF EXISTS Banco	0 row(s) affected
18	11:11:25	CREATE TABLE Banco (idBanco INT AUTO_INCREMENT PRIMARY KEY, nombreBanco VARCHAR(100))	0 row(s) affected
19	11:11:25	CREATE TABLE Cliente (nroDocumC VARCHAR(15) PRIMARY KEY, tipoDocumC VARCHAR(20), nombresC VARCHAR(100), paternoC VAR...	0 row(s) affected
20	11:11:25	CREATE TABLE Cuenta (idCuenta INT AUTO_INCREMENT PRIMARY KEY, nroCuenta VARCHAR(30), saldoCuenta DECIMAL(10,2), idBan...	0 row(s) affected
21	11:11:25	CREATE TABLE Empleado (nroDocumE VARCHAR(15) PRIMARY KEY, tipoDocumE VARCHAR(20), nombresE VARCHAR(100), paternoE V...	0 row(s) affected
22	11:11:25	CREATE TABLE Operacion (idOperacion INT AUTO_INCREMENT PRIMARY KEY, tipoOperacion VARCHAR(50), montoOperacion DECIMAL(...	0 row(s) affected

Diagrama:



3. Sistema de registro de asistencia del personal del INPE

CODIGO:

CREATE TABLE Area (

```
    idArea INT AUTO_INCREMENT PRIMARY KEY,
    nombreArea VARCHAR(100)
);

CREATE TABLE Empleado (
    nroDocumE VARCHAR(15) PRIMARY KEY,
    tipoDocumE VARCHAR(20),
    nombresE VARCHAR(100),
    paternoE VARCHAR(100),
    maternoE VARCHAR(100),
    idArea INT,
    FOREIGN KEY (idArea) REFERENCES Area(idArea)
);

CREATE TABLE Turno (
    idTurno INT AUTO_INCREMENT PRIMARY KEY,
    horaInicio TIME,
    horaFin TIME
);

CREATE TABLE Asistencia (
    idAsistencia INT AUTO_INCREMENT PRIMARY KEY,
    fechaA DATE,
```

horaEntrada TIME,

horaSalida TIME,

tardanza BOOLEAN,

nroDocumE VARCHAR(15),

idTurno INT,

FOREIGN KEY (nroDocumE) REFERENCES Empleado(nroDocumE),

FOREIGN KEY (idTurno) REFERENCES Turno(idTurno)

);

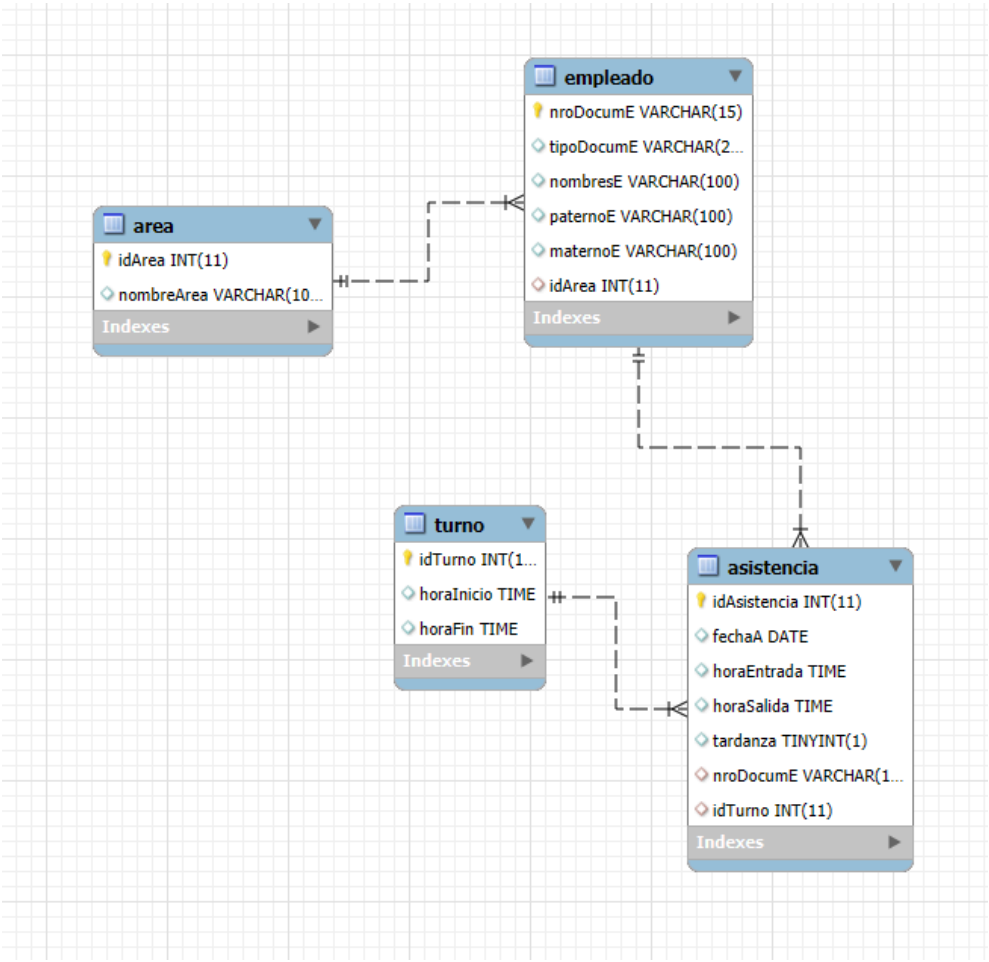
```sql
12  CREATE TABLE Area (
13      idArea INT AUTO_INCREMENT PRIMARY KEY,
14      nombreArea VARCHAR(100)
15  );
16
17  CREATE TABLE Empleado (
18      nroDocumE VARCHAR(15) PRIMARY KEY,
19      tipoDocumE VARCHAR(20),
20      nombresE VARCHAR(100),
21      paternoE VARCHAR(100),
22      maternoE VARCHAR(100),
23      idArea INT,
24      FOREIGN KEY (idArea) REFERENCES Area(idArea)
25  );
26
27  CREATE TABLE Turno (
28      idTurno INT AUTO_INCREMENT PRIMARY KEY,
29      horaInicio TIME,
30      horaFin TIME
31  );
32
33  CREATE TABLE Asistencia (
34      idAsistencia INT AUTO_INCREMENT PRIMARY KEY,
35      fechaA DATE,
36      horaEntrada TIME,
37      horaSalida TIME,
38      tardanza BOOLEAN,
39      nroDocumE VARCHAR(15),
40      idTurno INT,
41      FOREIGN KEY (nroDocumE) REFERENCES Empleado(nroDocumE),
```

Output

Action Output

#	Time	Action	Message
25	11:23:11	DROP TABLE IF EXISTS Area	0 row(s) affected
26	11:23:11	SET FOREIGN_KEY_CHECKS = 1	0 row(s) affected
27	11:23:11	CREATE TABLE Area (idArea INT AUTO_INCREMENT PRIMARY KEY, nombreArea VARCHAR(100))	0 row(s) affected
28	11:23:11	CREATE TABLE Empleado (nroDocumE VARCHAR(15) PRIMARY KEY, tipoDocumE VARCHAR(20), nombresE VARCHAR(100), paternoE V...	0 row(s) affected
29	11:23:11	CREATE TABLE Turno (idTurno INT AUTO_INCREMENT PRIMARY KEY, horaInicio TIME, horaFin TIME)	0 row(s) affected
30	11:23:11	CREATE TABLE Asistencia (idAsistencia INT AUTO_INCREMENT PRIMARY KEY, fechaA DATE, horaEntrada TIME, horaSalida TIME, t...	0 row(s) affected

DIAGRAMA



4. Sistema de registro de venta de platillos y bebidas en la pollería “Don Gallino”

CODIGO:

```
CREATE TABLE Cliente (

   nroDocumC VARCHAR(15) PRIMARY KEY,
```

```
    tipoDocumC VARCHAR(20),
    nombresC VARCHAR(100),
    paternoC VARCHAR(100),
    maternoC VARCHAR(100),
    celularC VARCHAR(15)
);

CREATE TABLE Categoria (
    idCategoria INT AUTO_INCREMENT PRIMARY KEY,
    descripcionCat VARCHAR(100)
);

CREATE TABLE Producto (
    idProducto INT AUTO_INCREMENT PRIMARY KEY,
    nombreProducto VARCHAR(100),
    precioReferencial DECIMAL(10,2),
    idCategoria INT,
    FOREIGN KEY (idCategoria) REFERENCES Categoria(idCategoria)
);

CREATE TABLE Comprobante (
    idComprobante INT AUTO_INCREMENT PRIMARY KEY,
    fechaHora DATETIME,
```

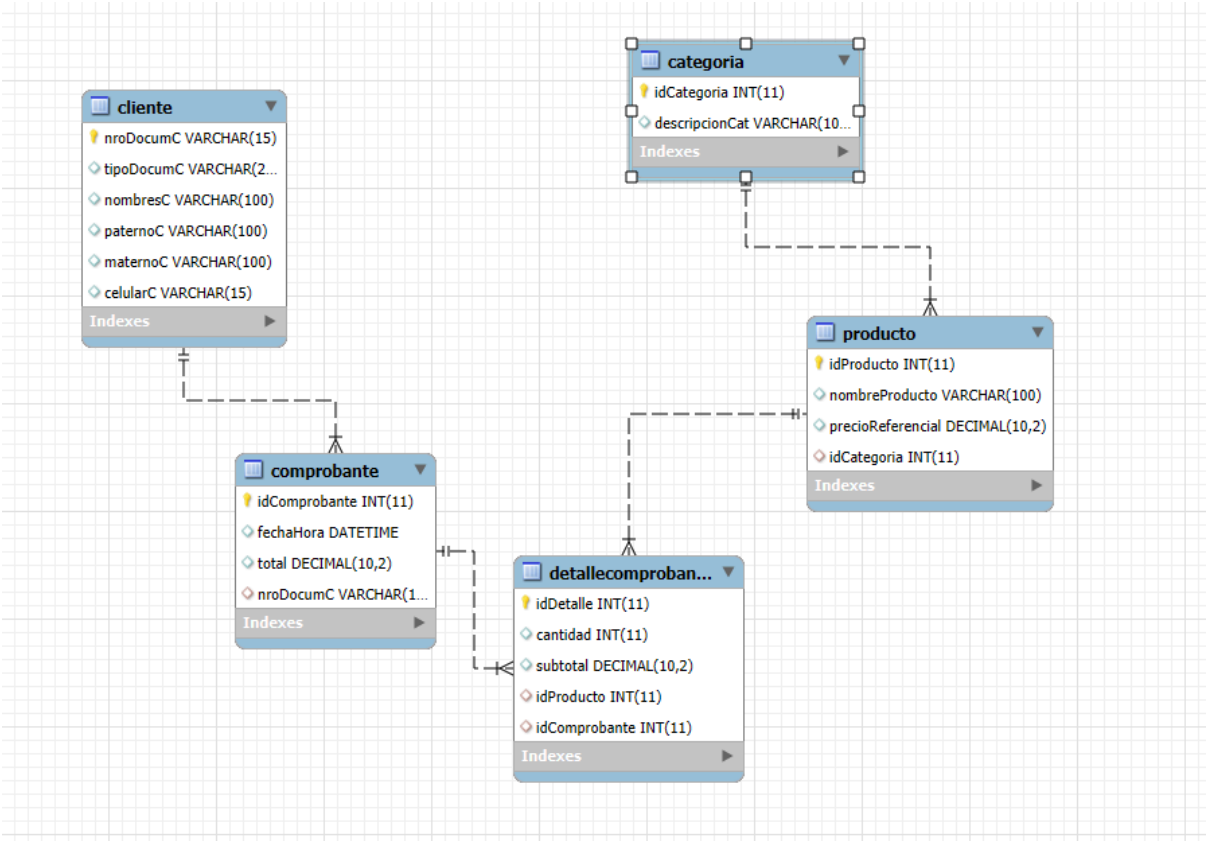
```
    total DECIMAL(10,2),
    nroDocumC VARCHAR(15),
    FOREIGN KEY (nroDocumC) REFERENCES Cliente(nroDocumC)
);

CREATE TABLE DetalleComprobante (
    idDetalle INT AUTO_INCREMENT PRIMARY KEY,
    cantidad INT,
    subtotal DECIMAL(10,2),
    idProducto INT,
    idComprobante INT,
    FOREIGN KEY (idProducto) REFERENCES Producto(idProducto),
    FOREIGN KEY (idComprobante) REFERENCES Comprobante(idComprobante)
);
```

```sql
    idCategoria INT AUTO_INCREMENT PRIMARY KEY,
    descripcionCat VARCHAR(100)
);

CREATE TABLE Producto (
    idProducto INT AUTO_INCREMENT PRIMARY KEY,
    nombreProducto VARCHAR(100),
    precioReferencial DECIMAL(10,2),
    idCategoria INT,
    FOREIGN KEY (idCategoria) REFERENCES Categoria(idCategoria)
);

CREATE TABLE Comprobante (
    idComprobante INT AUTO_INCREMENT PRIMARY KEY,
    fechaHora DATETIME,
    total DECIMAL(10,2),
    nroDocumC VARCHAR(15),
    FOREIGN KEY (nroDocumC) REFERENCES Cliente(nroDocumC)
);

CREATE TABLE DetalleComprobante (
    idDetalle INT AUTO_INCREMENT PRIMARY KEY,
    cantidad INT,
    subtotal DECIMAL(10,2),
    idProducto INT,
    idComprobante INT,
    FOREIGN KEY (idProducto) REFERENCES Producto(idProducto),
    FOREIGN KEY (idComprobante) REFERENCES Comprobante(idComprobante)
);
```

Output

Action Output

#	Time	Action	Message
72	11:53:15	DROP TABLE IF EXISTS Cliente	0 row(s) affected, 1 warning(s): 1051 Unknown table 'bd_polleria.cliente'
73	11:53:15	CREATE TABLE Cliente (nroDocumC VARCHAR(15) PRIMARY KEY, tipoDocumC VARCHAR(20), nombresC VARCHAR(100), paternoC VAR...	0 row(s) affected
74	11:53:15	CREATE TABLE Categoria (idCategoria INT AUTO_INCREMENT PRIMARY KEY, descripcionCat VARCHAR(100))	0 row(s) affected
75	11:53:15	CREATE TABLE Producto (idProducto INT AUTO_INCREMENT PRIMARY KEY, nombreProducto VARCHAR(100), precioReferencial DECIMA...	0 row(s) affected
76	11:53:15	CREATE TABLE Comprobante (idComprobante INT AUTO_INCREMENT PRIMARY KEY, fechaHora DATETIME, total DECIMAL(10,2), nroD...	0 row(s) affected
77	11:53:15	CREATE TABLE DetalleComprobante (idDetalle INT AUTO_INCREMENT PRIMARY KEY, cantidad INT, subtotal DECIMAL(10,2), idProduct...	0 row(s) affected


cliente
nroDocumC VARCHAR(15)
tipoDocumC VARCHAR(2...
nombresC VARCHAR(100)
paternoC VARCHAR(100)
maternoC VARCHAR(100)
celularC VARCHAR(15)
Indexes
categoria
idCategoria INT(11)
descripcionCat VARCHAR(10...
Indexes
producto
idProducto INT(11)
nombreProducto VARCHAR(100)
precioReferencial DECIMAL(10,2)
idCategoria INT(11)
Indexes
comprobante
idComprobante INT(11)
fechaHora DATETIME
total DECIMAL(10,2)
nroDocumC VARCHAR(1...
Indexes
detallecomproban...
idDetalle INT(11)
cantidad INT(11)
subtotal DECIMAL(10,2)
idProducto INT(11)
idComprobante INT(11)
Indexes

POSTGRESQL

1. Sistema de registro de transacciones bancarias a nivel nacional - Perú de la empresa Super Giros.

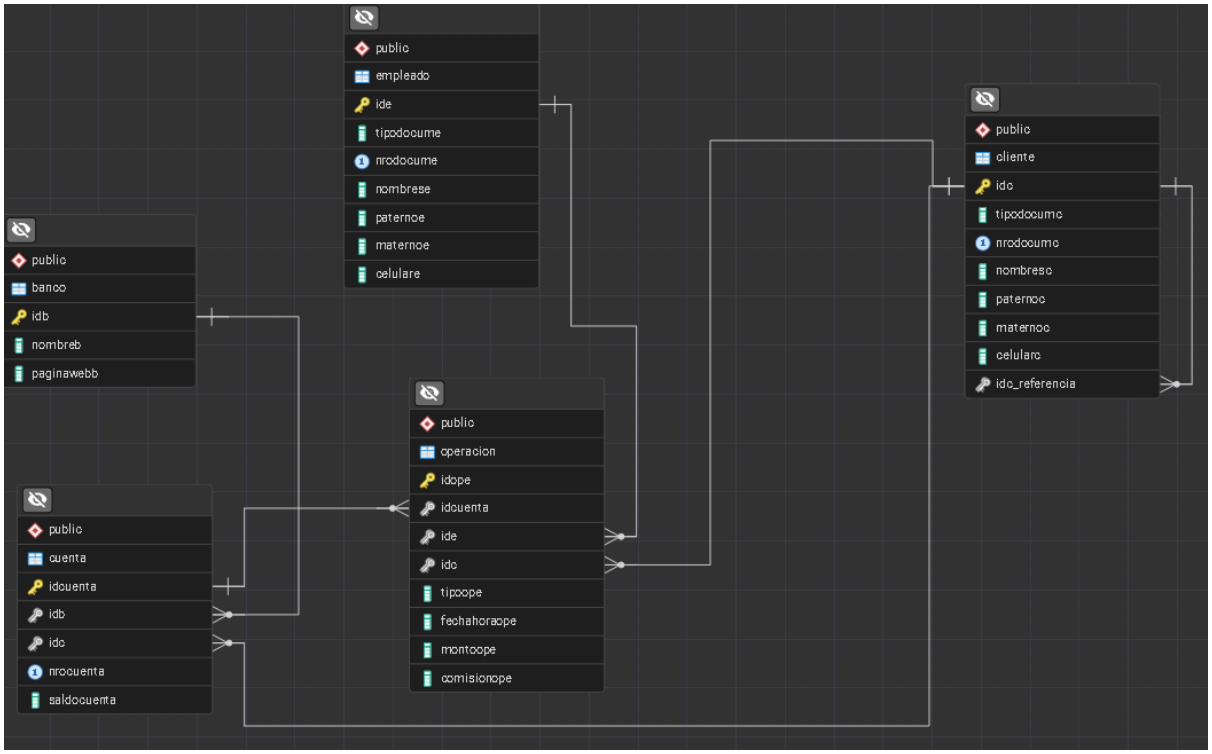
Código:

```
-- 1. Tabla Banco
CREATE TABLE Banco (
   idB SERIAL PRIMARY KEY,
   nombreB VARCHAR(100) NOT NULL,
   paginaWebB VARCHAR(150)
);

-- 2. Tabla Cliente (incluye relación recursiva)
CREATE TABLE Cliente (
   idC SERIAL PRIMARY KEY,
   tipoDocumC VARCHAR(20) NOT NULL,
   nroDocumC VARCHAR(20) UNIQUE NOT NULL,
   nombresC VARCHAR(100) NOT NULL,
   paternoC VARCHAR(50) NOT NULL,
   maternoC VARCHAR(50),
   celularC VARCHAR(15),
   idC_referencia INTEGER, -- El círculo (O) indica que es opcional
```

```
    CONSTRAINT fk_cliente_ref FOREIGN KEY (idC_referencia) REFERENCES
Cliente(idC)
);

-- 3. Tabla Empleado
CREATE TABLE Empleado (
    idE SERIAL PRIMARY KEY,
    tipoDocumE VARCHAR(20) NOT NULL,
    nroDocumE VARCHAR(20) UNIQUE NOT NULL,
    nombresE VARCHAR(100) NOT NULL,
    paternoE VARCHAR(50) NOT NULL,
    maternoE VARCHAR(50),
    celularE VARCHAR(15)
);

-- 4. Tabla Cuenta
-- Se relaciona con Banco y Cliente
CREATE TABLE Cuenta (
    idCuenta SERIAL PRIMARY KEY,
    idB INTEGER NOT NULL, -- Doble palito (||) desde Banco
    idC INTEGER NOT NULL, -- Doble palito (||) desde Cliente
    nroCuenta VARCHAR(30) UNIQUE NOT NULL,
    saldoCuenta DECIMAL(12, 2) NOT NULL DEFAULT 0.00,
```

```
    CONSTRAINT fk_cuenta_banco FOREIGN KEY (idB) REFERENCES Banco(idB),
    CONSTRAINT fk_cuenta_cliente FOREIGN KEY (idC) REFERENCES Cliente(idC)
);

-- 5. Tabla Operacion
-- Se relaciona con Cuenta, Empleado Y Cliente (corregido)
CREATE TABLE Operacion (
    idOpe SERIAL PRIMARY KEY,
    idCuenta INTEGER NOT NULL, -- Doble palito (||) desde Cuenta
    idE INTEGER NOT NULL,      -- Doble palito (||) desde Empleado
    idC INTEGER NOT NULL,      -- Doble palito (||) desde Cliente
    tipoOpe VARCHAR(50) NOT NULL,
    fechaHoraOpe TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    montoOpe DECIMAL(12, 2) NOT NULL,
    comisionOpe DECIMAL(12, 2) NOT NULL DEFAULT 0.00,
    CONSTRAINT fk_ope_cuenta FOREIGN KEY (idCuenta) REFERENCES
Cuenta(idCuenta),
    CONSTRAINT fk_ope_empleado FOREIGN KEY (idE) REFERENCES
Empleado(idE),
    CONSTRAINT fk_ope_cliente FOREIGN KEY (idC) REFERENCES Cliente(idC)
);
```

Diagrama:



2. Sistema de registro de asistencia del personal del INPE.

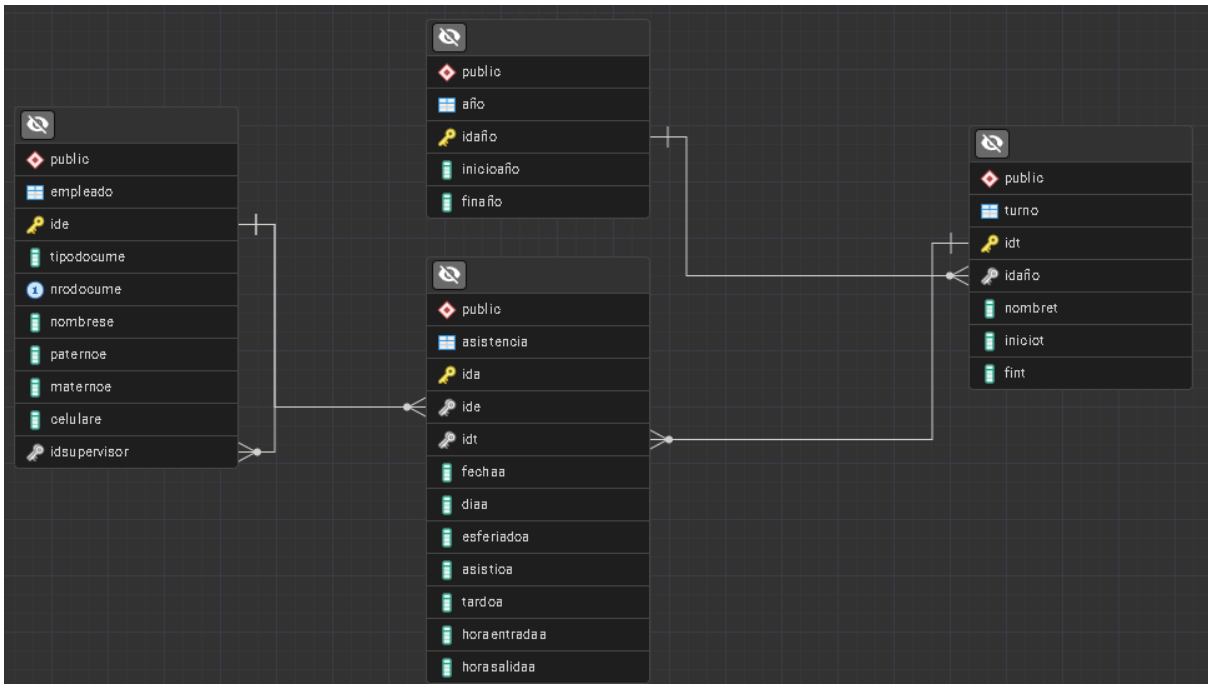
Código:

```
-- 1. Tabla Año
CREATE TABLE Año (
    idAño SERIAL PRIMARY KEY,
    inicioAño DATE NOT NULL,
    finAño DATE NOT NULL
);

-- 2. Tabla Empleado (Incluye la relación recursiva Supervisor/Supervisado)
CREATE TABLE Empleado (
    idE SERIAL PRIMARY KEY,
```

```
    tipoDocumE VARCHAR(20) NOT NULL,
    nroDocumE VARCHAR(20) UNIQUE NOT NULL,
    nombresE VARCHAR(100) NOT NULL,
    paternoE VARCHAR(50) NOT NULL,
    maternoE VARCHAR(50),
    celularE VARCHAR(15),
    idSupervisor INTEGER, -- FK para la relación recursiva (puede ser NULL para el jefe máximo)
    CONSTRAINT fk_empleado_supervisor FOREIGN KEY (idSupervisor) REFERENCES Empleado(idE)
);

-- 3. Tabla Turno
-- Se relaciona con Año (|| ------ |<)
CREATE TABLE Turno (
    idT SERIAL PRIMARY KEY,
    idAño INTEGER NOT NULL, -- Llave foránea que viene de Año
    nombreT VARCHAR(50) NOT NULL,
    inicioT TIME NOT NULL,
    finT TIME NOT NULL,
    CONSTRAINT fk_turno_anio FOREIGN KEY (idAño) REFERENCES Año(idAño)
);
```

```
-- 4. Tabla Asistencia
-- Se relaciona con Turno (|| ------ |<) y Empleado (|| ------ |<)
CREATE TABLE Asistencia (
    idA SERIAL PRIMARY KEY,
    idE INTEGER NOT NULL, -- Llave foránea que viene de Empleado
    idT INTEGER NOT NULL, -- Llave foránea que viene de Turno
    fechaA DATE NOT NULL,
    diaA VARCHAR(20),
    esFeriadoA BOOLEAN DEFAULT FALSE,
    asistioA BOOLEAN DEFAULT FALSE,
    tardoA BOOLEAN DEFAULT FALSE,
    horaEntradaA TIME,
    horaSalidaA TIME,
    CONSTRAINT fk_asistencia_empleado FOREIGN KEY (idE) REFERENCES
Empleado(idE),
    CONSTRAINT fk_asistencia_turno FOREIGN KEY (idT) REFERENCES Turno(idT)
);
```

Diagrama:



3. Sistema de registro de venta de platillos y bebidas en la pollería "Don Gallino"

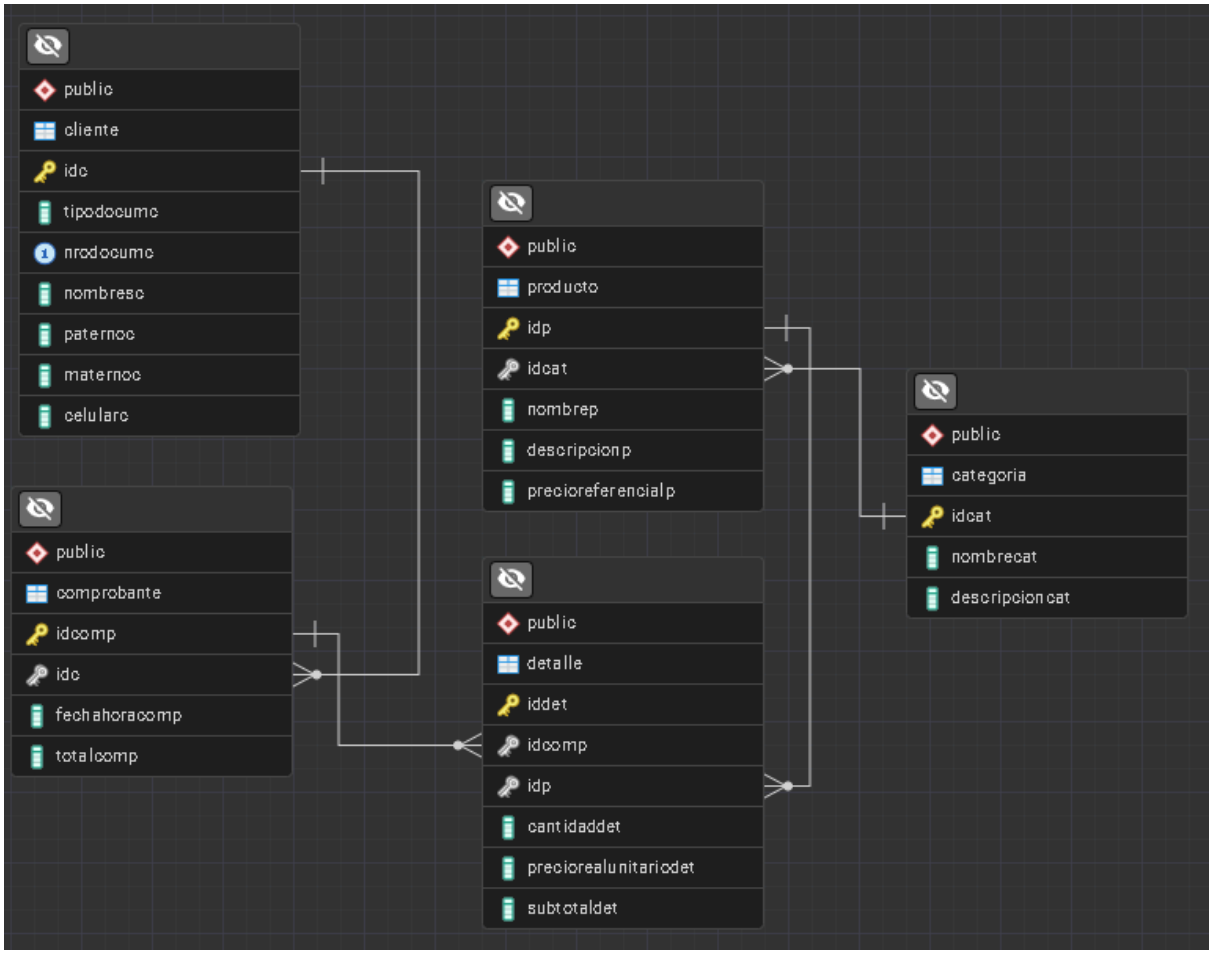
Código:

```
-- 1. Tabla Cliente (Tabla independiente)
CREATE TABLE Cliente (
    idC SERIAL PRIMARY KEY,
    tipoDocumC VARCHAR(20) NOT NULL,
    nroDocumC VARCHAR(20) UNIQUE NOT NULL,
    nombresC VARCHAR(100) NOT NULL,
    paternoC VARCHAR(50) NOT NULL,
    maternoC VARCHAR(50),
    celularC VARCHAR(15)
);
```

```
-- 2. Tabla Categoría (Tabla independiente)
CREATE TABLE Categoria (
    idCat SERIAL PRIMARY KEY,
    nombreCat VARCHAR(100) NOT NULL,
    descripcionCat TEXT
);

-- 3. Tabla Producto
-- Se relaciona con Categoría (|| ------ |<)
CREATE TABLE Producto (
    idP SERIAL PRIMARY KEY,
    idCat INTEGER NOT NULL, -- Llave foránea de Categoría (Obligatoria por el ||)
    nombreP VARCHAR(150) NOT NULL,
    descripcionP TEXT,
    precioReferencialP DECIMAL(10, 2) NOT NULL,
    CONSTRAINT fk_producto_categoria FOREIGN KEY (idCat) REFERENCES
Categoria(idCat)
);

-- 4. Tabla Comprobante
-- Se relaciona con Cliente (|| ------ |<)
CREATE TABLE Comprobante (
    idComp SERIAL PRIMARY KEY,
```

```
    idC INTEGER NOT NULL, -- Llave foránea de Cliente (Obligatoria por el ||)
    fechaHoraComp TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    totalComp DECIMAL(12, 2) NOT NULL DEFAULT 0.00,
    CONSTRAINT fk_comprobante_cliente FOREIGN KEY (idC) REFERENCES
Cliente(idC)
);

-- 5. Tabla Detalle
-- Se relaciona con Comprobante (|| ------ |<) y Producto (|| ------ |<)
CREATE TABLE Detalle (
    idDet SERIAL PRIMARY KEY,
    idComp INTEGER NOT NULL, -- Llave foránea de Comprobante
    idP INTEGER NOT NULL,    -- Llave foránea de Producto
    cantidadDet INTEGER NOT NULL,
    precioRealUnitarioDet DECIMAL(10, 2) NOT NULL,
    subtotalDet DECIMAL(12, 2) NOT NULL,
    CONSTRAINT fk_detalle_comprobante FOREIGN KEY (idComp) REFERENCES
Comprobante(idComp),
    CONSTRAINT fk_detalle_producto FOREIGN KEY (idP) REFERENCES
Producto(idP)
);
```

Diagrama:



4. Sistema de registro de entrada y salida de electrodomésticos "Electro Track"

Código:

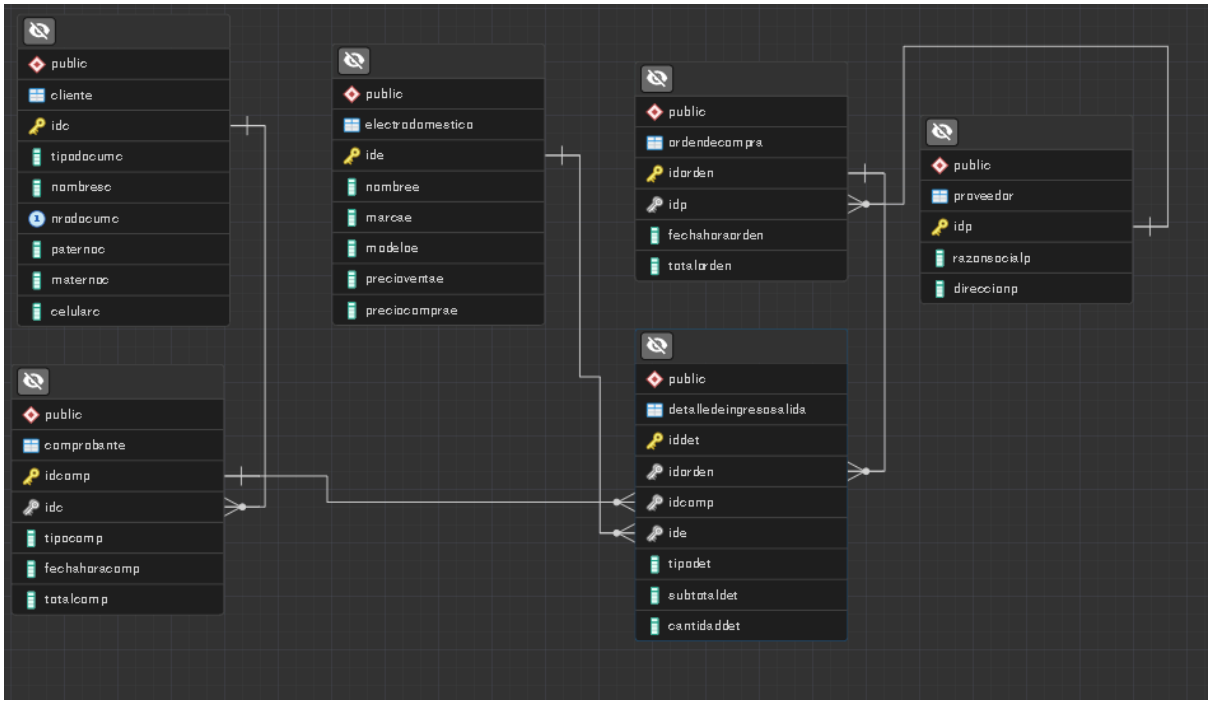
```
-- 1. Tabla Proveedor (Independiente)

CREATE TABLE Proveedor (

    idP SERIAL PRIMARY KEY,

    razonSocialP VARCHAR(150) NOT NULL,

    direccionP VARCHAR(200)

);


-- 2. Tabla Cliente (Independiente)
```

```
CREATE TABLE Cliente (
    idC SERIAL PRIMARY KEY,
    tipoDocumC VARCHAR(20) NOT NULL,
    nombresC VARCHAR(100) NOT NULL,
    nroDocumC VARCHAR(20) UNIQUE NOT NULL,
    paternoC VARCHAR(50) NOT NULL,
    maternoC VARCHAR(50),
    celularC VARCHAR(15)
);

-- 3. Tabla Electrodomestico (Independiente)
CREATE TABLE Electrodomestico (
    idE SERIAL PRIMARY KEY,
    nombreE VARCHAR(100) NOT NULL,
    marcaE VARCHAR(50),
    modeloE VARCHAR(50),
    precioVentaE DECIMAL(10, 2) NOT NULL,
    precioCompraE DECIMAL(10, 2) NOT NULL
);

-- 4. Tabla OrdenDeCompra
-- Se relaciona con Proveedor (|| ------ |<)
CREATE TABLE OrdenDeCompra (
```

```
    idOrden SERIAL PRIMARY KEY,
    idP INTEGER NOT NULL, -- Obligatorio (||)
    fechaHoraOrden TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    totalOrden DECIMAL(12, 2) NOT NULL DEFAULT 0.00,
    CONSTRAINT fk_orden_proveedor FOREIGN KEY (idP) REFERENCES
Proveedor(idP)
);

-- 5. Tabla Comprobante
-- Se relaciona con Cliente (|| ------ |<)
CREATE TABLE Comprobante (
    idComp SERIAL PRIMARY KEY,
    idC INTEGER NOT NULL, -- Obligatorio (||)
    tipoComp VARCHAR(50) NOT NULL,
    fechaHoraComp TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    totalComp DECIMAL(12, 2) NOT NULL DEFAULT 0.00,
    CONSTRAINT fk_comprobante_cliente FOREIGN KEY (idC) REFERENCES
Cliente(idC)
);

-- 6. Tabla detalleDeIngresoSalida
-- Se relaciona con OrdenDeCompra (O), Comprobante (O) y Electrodomestico (||)
CREATE TABLE detalleDeIngresoSalida (
```

```
idDet SERIAL PRIMARY KEY,
idOrden INTEGER, -- Opcional (O): No lleva NOT NULL
idComp INTEGER,  -- Opcional (O): No lleva NOT NULL
idE INTEGER NOT NULL, -- Obligatorio (||)
tipoDet VARCHAR(50) NOT NULL, -- Para identificar si es 'INGRESO' o 'SALIDA'
subtotalDet DECIMAL(12, 2) NOT NULL,
cantidadDet INTEGER NOT NULL,
CONSTRAINT fk_detalle_orden FOREIGN KEY (idOrden) REFERENCES OrdenDeCompra(idOrden),
CONSTRAINT fk_detalle_comprobante FOREIGN KEY (idComp) REFERENCES Comprobante(idComp),
CONSTRAINT fk_detalle_electrodomestico FOREIGN KEY (idE) REFERENCES Electrodomestico(idE)
);
```

Diagrama:



5. Sistema de registro de asistencia de las enfermeras a la clínica Mac Salud

Código:

```
-- 1. Tabla Enfermero (Independiente)
CREATE TABLE Enfermero (
    idE SERIAL PRIMARY KEY,
    tipoDocumE VARCHAR(20) NOT NULL,
    nroDocumE VARCHAR(20) UNIQUE NOT NULL,
    nombresE VARCHAR(100) NOT NULL,
    paternoE VARCHAR(50) NOT NULL,
    maternoE VARCHAR(50),
    fechaNacimientoE DATE NOT NULL,
    generoE VARCHAR(20)
);
```

```
-- 2. Tabla Calendario (Independiente)
CREATE TABLE Calendario (
    idC SERIAL PRIMARY KEY,
    inicioC DATE NOT NULL,
    finC DATE NOT NULL,
    nombreC VARCHAR(100) NOT NULL
);

-- 3. Tabla Ambiente (Independiente)
CREATE TABLE Ambiente (
    idAmb SERIAL PRIMARY KEY,
    nroAmb VARCHAR(20) NOT NULL,
    ubicacionAmb VARCHAR(150) NOT NULL
);

-- 4. Tabla Asistencia
-- Se relaciona con Enfermero (||), Calendario (||) y Ambiente (||)
CREATE TABLE Asistencia (
    idA SERIAL PRIMARY KEY,
    idE INTEGER NOT NULL,    -- Llave foránea de Enfermero
    idC INTEGER NOT NULL,    -- Llave foránea de Calendario
    idAmb INTEGER NOT NULL,  -- Llave foránea de Ambiente
```

fechaA DATE NOT NULL,

diaA VARCHAR(20) NOT NULL,

asistioA BOOLEAN DEFAULT FALSE,

tardoA BOOLEAN DEFAULT FALSE,

horaEntradaA TIME,

horaSalidaA TIME,

justificacionA TEXT,

esDescansoA BOOLEAN DEFAULT FALSE,

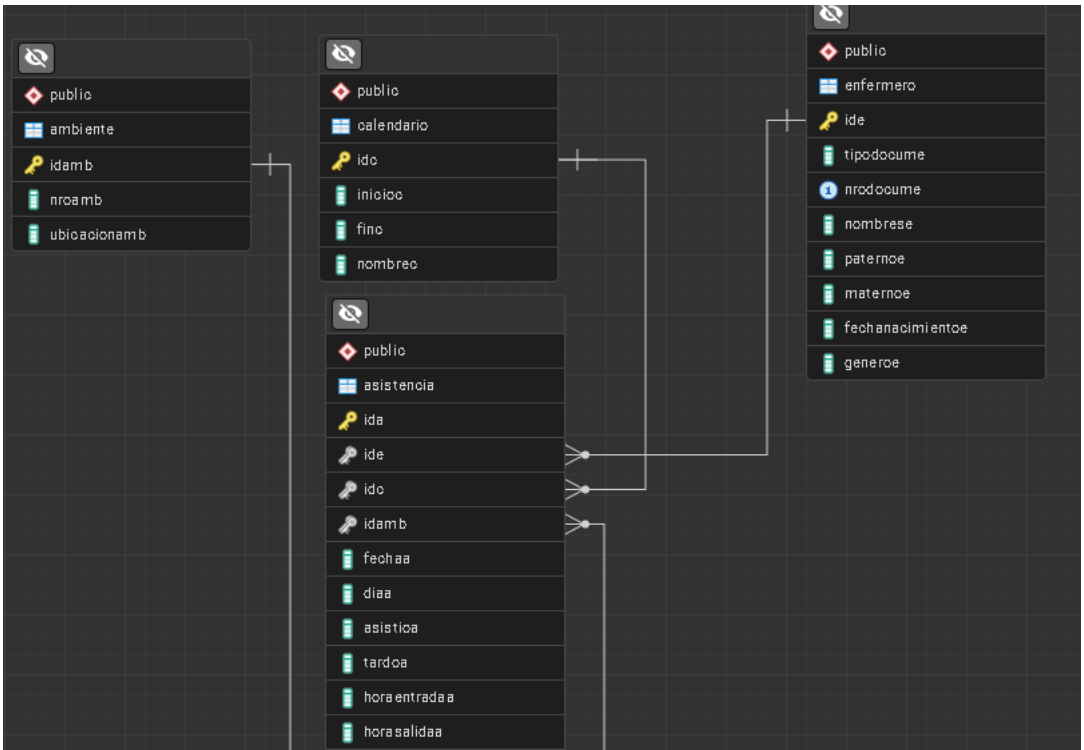
CONSTRAINT fk_asistencia_enfermero FOREIGN KEY (idE) REFERENCES Enfermero(idE),

CONSTRAINT fk_asistencia_calendario FOREIGN KEY (idC) REFERENCES Calendario(idC),

CONSTRAINT fk_asistencia_ambiente FOREIGN KEY (idAmb) REFERENCES Ambiente(idAmb)

);

Diagrama:



6. Sistema de facturación electrónica de la pollería Los Ángeles.

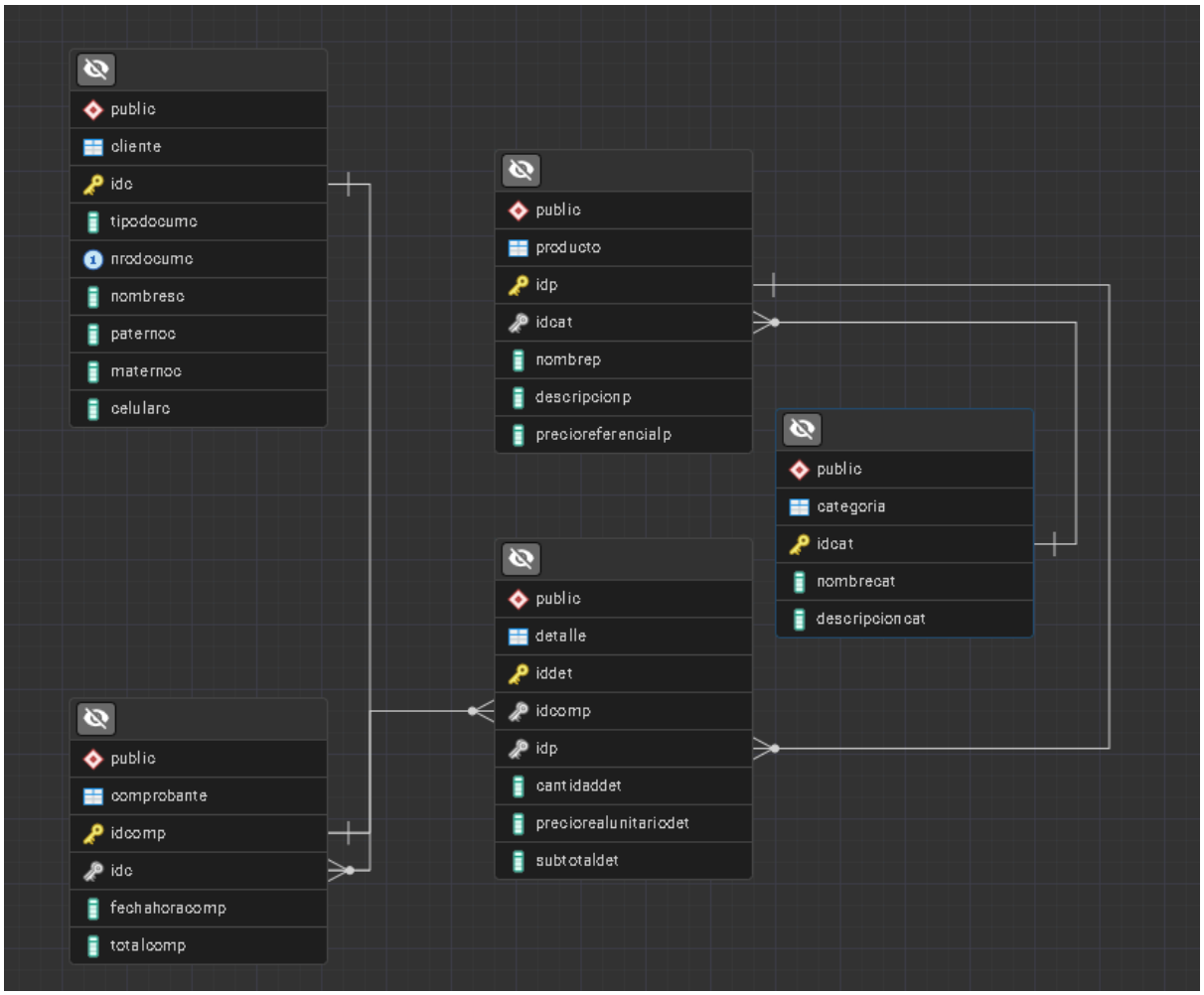
Código:

```
-- 1. Tabla Cliente (Independiente)
CREATE TABLE Cliente (
    idC SERIAL PRIMARY KEY,
    tipoDocumC VARCHAR(20) NOT NULL,
    nroDocumC VARCHAR(20) UNIQUE NOT NULL,
    nombresC VARCHAR(100) NOT NULL,
    paternoC VARCHAR(50) NOT NULL,
    maternoC VARCHAR(50),
    celularC VARCHAR(15)
);

-- 2. Tabla Categoria (Independiente)
CREATE TABLE Categoria (
    idCat SERIAL PRIMARY KEY,
    nombreCat VARCHAR(100) NOT NULL,
    descripcionCat TEXT
);

-- 3. Tabla Producto
-- Se relaciona con Categoria (|| ------ |<)
CREATE TABLE Producto (
```

```
    idP SERIAL PRIMARY KEY,
    idCat INTEGER NOT NULL, -- Obligatorio (||)
    nombreP VARCHAR(150) NOT NULL,
    descripcionP TEXT,
    precioReferencialP DECIMAL(10, 2) NOT NULL,
    CONSTRAINT fk_producto_categoria FOREIGN KEY (idCat) REFERENCES
Categoria(idCat)
);

-- 4. Tabla Comprobante
-- Se relaciona con Cliente (|| ------ |<)
CREATE TABLE Comprobante (
    idComp SERIAL PRIMARY KEY,
    idC INTEGER NOT NULL, -- Obligatorio (||)
    fechaHoraComp TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    totalComp DECIMAL(12, 2) NOT NULL DEFAULT 0.00,
    CONSTRAINT fk_comprobante_cliente FOREIGN KEY (idC) REFERENCES
Cliente(idC)
);

-- 5. Tabla Detalle
-- Se relaciona con Comprobante (|| ------ |<) y Producto (|| ------ |<)
CREATE TABLE Detalle (
```

```
    idDet SERIAL PRIMARY KEY,
    idComp INTEGER NOT NULL, -- Obligatorio (||)
    idP INTEGER NOT NULL,    -- Obligatorio (||)
    cantidadDet INTEGER NOT NULL,
    precioRealUnitarioDet DECIMAL(10, 2) NOT NULL,
    subtotalDet DECIMAL(12, 2) NOT NULL,
    CONSTRAINT fk_detalle_comprobante FOREIGN KEY (idComp) REFERENCES
Comprobante(idComp),
    CONSTRAINT fk_detalle_producto FOREIGN KEY (idP) REFERENCES
Producto(idP)
);
```

Diagrama:



7. Registro de asistencia de clientes del gimnasio 'Titanium'

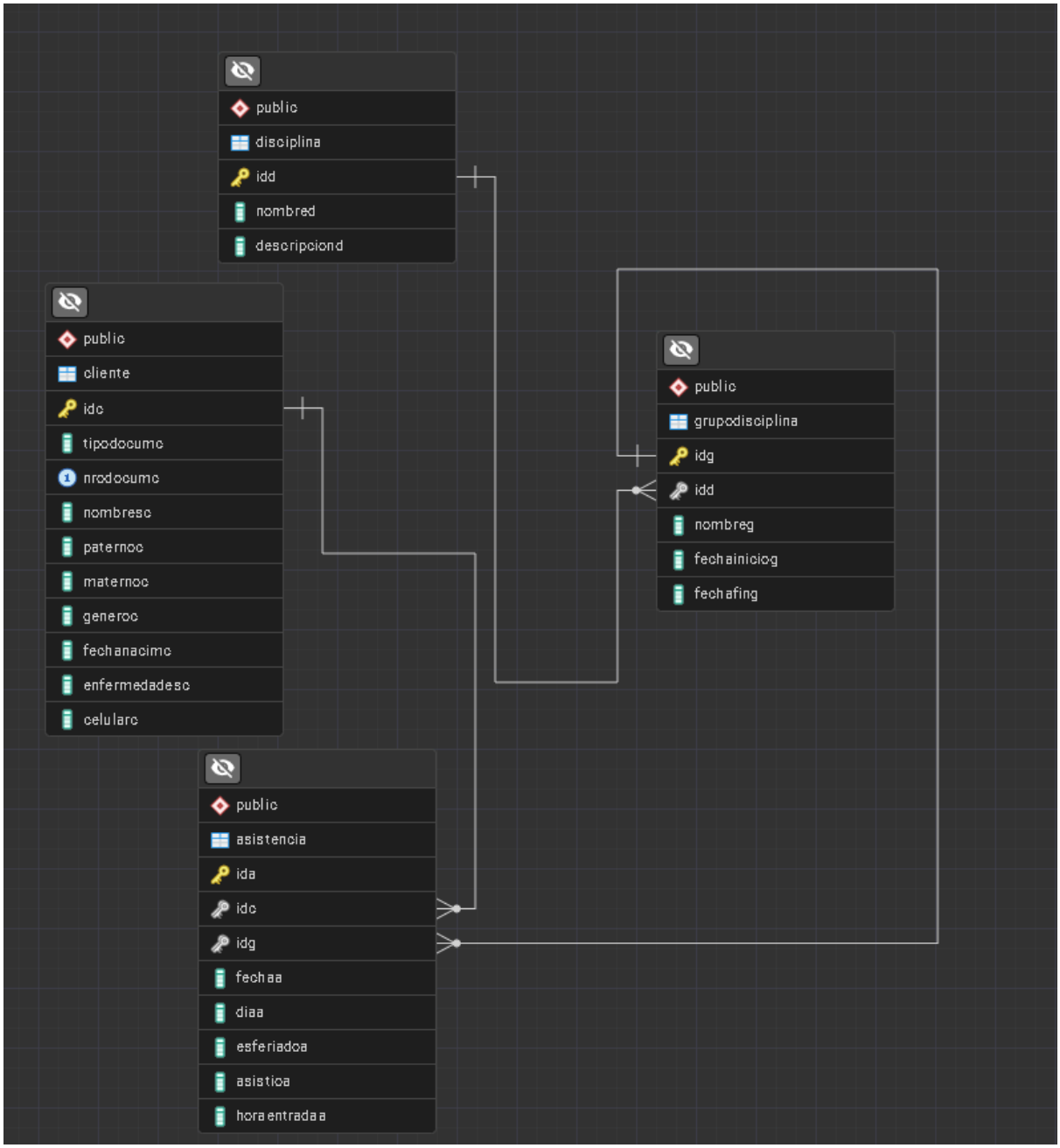
Código:

```
-- 1. Tabla Cliente (Independiente)

CREATE TABLE Cliente (

    idC SERIAL PRIMARY KEY,

    tipoDocumC VARCHAR(20) NOT NULL,

    nroDocumC VARCHAR(20) UNIQUE NOT NULL,

    nombresC VARCHAR(100) NOT NULL,

    paternoC VARCHAR(50) NOT NULL,
```

```
    maternoC VARCHAR(50),
    generoC VARCHAR(20),
    fechaNacimC DATE,
    enfermedadesC TEXT, -- Corregido de "enfremedadesC"
    celularC VARCHAR(15)
);

-- 2. Tabla Disciplina (Independiente)
CREATE TABLE Disciplina (
    idD SERIAL PRIMARY KEY,
    nombreD VARCHAR(100) NOT NULL,
    descripcionD TEXT
);

-- 3. Tabla GrupoDisciplina
-- Se relaciona con Disciplina (|| ------ |<)
CREATE TABLE GrupoDisciplina (
    idG SERIAL PRIMARY KEY,
    idD INTEGER NOT NULL, -- Llave foránea obligatoria (||)
    nombreG VARCHAR(100) NOT NULL,
    fechaInicioG DATE NOT NULL,
    fechaFinG DATE NOT NULL,
```

```
    CONSTRAINT fk_grupo_disciplina FOREIGN KEY (idD) REFERENCES Disciplina(idD)
);

-- 4. Tabla Asistencia
-- Se relaciona con Cliente (|| ------ |<) y GrupoDisciplina (|| ------ |<)
CREATE TABLE Asistencia (
    idA SERIAL PRIMARY KEY,
    idC INTEGER NOT NULL, -- Llave foránea obligatoria (||)
    idG INTEGER NOT NULL, -- Llave foránea obligatoria (||)
    fechaA DATE NOT NULL,
    diaA VARCHAR(20) NOT NULL,
    esFeriadoA BOOLEAN DEFAULT FALSE, -- Corregido de "esferiadoA"
    asistioA BOOLEAN DEFAULT FALSE,
    horaEntradaA TIME,
    CONSTRAINT fk_asistencia_cliente FOREIGN KEY (idC) REFERENCES Cliente(idC),
    CONSTRAINT fk_asistencia_grupo FOREIGN KEY (idG) REFERENCES GrupoDisciplina(idG)
);
```

Diagrama:


public
disciplina
idd
nombred
descripciond
public
cliente
idc
tipodocumc
nrodocumc
nombresc
paternoc
maternoc
generoc
fechanacimc
enfermedadesc
celularc
public
grupodisciplina
idg
idd
nombreg
fechainiciog
fechafing
public
asistencia
ida
idc
idg
fechaa
diaa
esferiadoa
asistioa
horaentradaa

Diferencias encontradas en la sintaxis y funcionalidades de los SGBD

Durante la implementación del modelo físico en Oracle, SQL Server, MySQL y PostgreSQL, se identificaron diversas diferencias en la sintaxis y en algunas funcionalidades propias de cada sistema gestor de bases de datos. Aunque todos permiten crear tablas, definir claves primarias, claves foráneas y restricciones de integridad, la forma de hacerlo varía según el SGBD utilizado.

En Oracle, se emplearon tipos de datos como VARCHAR2, NUMBER y DATE, lo cual evidencia una sintaxis distinta respecto a los demás gestores. Asimismo, para eliminar tablas previamente existentes se utilizó la instrucción DROP TABLE ... CASCADE CONSTRAINTS, lo que permite borrar tablas considerando sus restricciones relacionadas. En este sistema, la definición de claves foráneas y restricciones se integra directamente en la creación de la tabla.

En SQL Server, se observa una sintaxis diferente en la administración de bases de datos, ya que se utiliza USE, DB_ID, OBJECT_ID y GO, además de la creación explícita de la base de datos antes de construir las tablas. También se emplea IDENTITY(1,1) para generar identificadores automáticos. Asimismo, en este gestor se aprecia el uso de restricciones agregadas tanto dentro de la creación de la tabla como mediante instrucciones ALTER TABLE, incluyendo restricciones UNIQUE y CHECK.

En MySQL, la generación automática de claves primarias se realizó mediante AUTO_INCREMENT, lo cual constituye una diferencia importante frente a los otros gestores. Su sintaxis de creación de tablas resulta más directa y compacta, con la definición de claves foráneas dentro de cada tabla. Además, en algunos casos se utilizaron atributos documentales, como el número de documento, como clave primaria principal en ciertas entidades.

En PostgreSQL, se evidencia el uso de SERIAL para la generación automática de identificadores, lo cual representa otra diferencia relevante respecto a Oracle, SQL Server y MySQL. También se observa el uso de restricciones como UNIQUE y claves foráneas declaradas directamente dentro de la tabla, manteniendo la lógica relacional del modelo, pero adaptada a la sintaxis propia del motor.

En conjunto, puede afirmarse que las principales diferencias encontradas entre los SGBD estuvieron en el manejo de tipos de datos, la forma de generar identificadores automáticos, el uso de instrucciones para eliminación y creación de objetos, y la manera de declarar restricciones. A pesar de estas variaciones, fue posible implementar el mismo modelo físico en todos los gestores, realizando las adaptaciones necesarias sin alterar la estructura lógica del diseño.

Decisiones tomadas durante el proceso de implementación

Durante el proceso de implementación del modelo físico en Oracle, SQL Server, MySQL y PostgreSQL, se tomaron diversas decisiones con el fin de mantener la integridad de los datos, la coherencia del diseño y la compatibilidad del modelo en los distintos sistemas gestores de bases de datos.

En primer lugar, se decidió conservar la estructura lógica de cada modelo conceptual al momento de transformarlo en modelo físico, respetando las entidades, atributos y relaciones originalmente planteadas. Esto permitió que la implementación en los distintos SGBD mantuviera la misma base de diseño, realizando únicamente los ajustes necesarios en la sintaxis y en los tipos de datos según el motor utilizado.

Asimismo, se optó por definir claves primarias en todas las tablas principales para garantizar la identificación única de cada registro. De la misma forma, se implementaron claves foráneas en las tablas relacionadas con el propósito de preservar la integridad referencial entre las entidades y asegurar la correcta vinculación de los datos.

Otra decisión importante fue adaptar los tipos de datos a las características propias de cada sistema gestor. Por ejemplo, en Oracle se emplearon tipos como `VARCHAR2` y `NUMBER`, mientras que en SQL Server, MySQL y PostgreSQL se utilizaron equivalentes acordes con su sintaxis. Esta decisión permitió representar adecuadamente la misma información en cada plataforma sin alterar la naturaleza de los atributos.

También se decidió utilizar mecanismos de generación automática de identificadores cuando el SGBD lo permitía y el modelo lo requería. En SQL Server se empleó `IDENTITY(1,1)`, en MySQL se utilizó `AUTO_INCREMENT` y en PostgreSQL `SERIAL`. Esto facilitó el registro de nuevos datos y simplificó el control de claves primarias numéricas en varias tablas.

Además, se incorporaron restricciones como `UNIQUE` y `CHECK` en aquellos atributos que requerían control adicional, con el objetivo de evitar duplicidad de información y validar condiciones específicas dentro de la base de datos. Esto fortaleció la consistencia de los datos y permitió una implementación más segura del modelo físico.

Finalmente, se decidió organizar la implementación por sistema gestor de base de datos, separando los scripts según Oracle, SQL Server, MySQL y PostgreSQL. Esta decisión facilitó la comprensión del trabajo desarrollado, permitió identificar con mayor claridad las diferencias de sintaxis entre motores y ayudó a mantener un mejor orden en la documentación del proyecto.

5. CONCLUSIONES

1. **El diseño lógico manda:** Comprobamos que si el diagrama físico está bien hecho (con sus llaves primarias y foráneas bien definidas), pasarlo a código es mucho más fácil. La estructura fundamental de la base de datos se mantiene sin importar si usamos Oracle, SQL Server, MySQL o PostgreSQL.
2. **El orden de las tablas es clave:** Aprendimos que no se puede crear o borrar las tablas en cualquier orden. Siempre se debe respetar la jerarquía de las llaves foráneas. Las tablas "padre" se crean primero y las tablas "hijas" se crean después para no romper la integridad de los datos.
3. **Diferencias en el código:** Notamos que MySQL y PostgreSQL son más directos para crear campos autoincrementales (AUTO_INCREMENT y SERIAL), mientras que Oracle y SQL Server requieren instrucciones un poco más específicas, pero igual de efectivas.
4. **Manejo de herramientas:** Hacer este ejercicio en grupo y en distintos gestores nos ayudó a entender que el lenguaje SQL estándar tiene variaciones. Como futuros ingenieros, esto nos prepara para saber adaptarnos rápidamente a la base de datos que utilice la empresa donde vayamos a trabajar.

Disponibilidad del Código Fuente *(Alineado a la izquierda y en negrita)*

Para garantizar la transparencia, la reproducibilidad y el correcto control de versiones del presente trabajo, los scripts completos de Lenguaje de Definición de Datos (DDL) desarrollados para cada gestor de bases de datos han sido alojados en repositorios públicos de la plataforma GitHub. A continuación, se detallan los enlaces de acceso directo al código fuente publicado por cada integrante del equipo:

- **PostgreSQL:** Olivera Ochoa Paul Claus. Recuperado de: [Pega aquí enlace de GitHub, ej: https://github.com/tu-usuario/tu-repo]
- **Oracle:** Carpio Delgado Alejandro Eduardo. Recuperado de: [Pega aquí enlace de GitHub]
- **SQL Server:** Haro Rosales Joaquín. Recuperado de: [Pega aquí enlace de GitHub de tu compañero]
- **MySQL:** Molero Zegarra Thiago Valentino. Recuperado de: [Pega aquí el enlace de GitHub de tu compañero]